SELECT … FROM

WHERE

COMPLETE COURSE · MYSQL 8 · HANDS-ON

GROUP BY

SQL

MASTERCLASS

JOIN … ON

From your very first **SELECT** to production-ready thinking — deep explanations of every core topic, real examples rebuilt from seven hands-on lecture notebooks, and three guided projects you can run on your own MySQL server today.

16	3	7	23
CHAPTERS	PROJECTS	LECTURE NOTEBOOKS	PAGES

SCROLL ↓ · START WITH THE INDEX →

CREATE INDEX

📘 Built from the **MySql Notebooks** practice repository

Beginner → Advanced · September 2026 · v1.0

CASE WHEN

NAVIGATION GUIDE MAP

Course Index

Five parts, sixteen chapters, three projects. Every chapter lives on its own page — click any row to jump straight there. Page numbers refer to guide pages (sections), not printed sheets.

PART I — FOUNDATIONS

01 SQL & Databases 101 — how MySQL thinks ..... p. 04

02 Creating Databases & Tables — types & constraints ..... p. 05

03 Modifying & Destroying — ALTER, DELETE vs TRUNCATE vs DROP ..... p. 06

PART II — WORKING WITH DATA

04 Inserting Data — INSERT done right ..... p. 07

05 Reading Data — SELECT & the logical query order ..... p. 08

06 Filtering — WHERE, IN, LIKE, NULL logic ..... p. 09

07 Sorting & Paging — ORDER BY, LIMIT, OFFSET ..... p. 10

PART III — ANALYZING DATA

08 Aggregates — COUNT, SUM, MIN, MAX, AVG & NULLs ..... p. 11

09 Grouping — GROUP BY, HAVING & duplicate hunters ..... p. 12

10 Subqueries — queries inside queries ..... p. 13

PART IV — RELATIONSHIPS

11 Relationships & Foreign Keys — 1:1, 1:N, M:N ..... p. 14

12 Joins — INNER, LEFT, RIGHT, CROSS & anti-joins ..... p. 15

PART V — POWER TOOLS

13 Transactions — COMMIT, ROLLBACK & ACID ..... p. 16

14 Views — saved queries as virtual tables p. 17

15 CASE & Conditionals — tiers, pivots, IFNULL p. 18

16 Indexes & EXPLAIN — making queries fast p. 19

PROJECTS

P1 E-Commerce Analytics — revenue, tiers & top customers p. 20

P2 Student Enrollment System — many-to-many done properly p. 21

P3 Performance Tune-up — EXPLAIN before & after p. 22

REFERENCE

★ One-Page Cheatsheet & What to Learn Next p. 23

How this guide maps to your lectures

LECTURE NOTEBOOK	WHAT IT PRACTICED	DEEP-DIVED IN CHAPTERS
Lecture 1	CREATE TABLE, INSERT, UPDATE, DELETE, TRUNCATE, DROP	Ch 2 · 3 · 4
Lecture 2	SELECT, WHERE, IN, LIKE, LIMIT / OFFSET, bulk seeding	Ch 4 · 5 · 6 · 7
Lecture 3	Combined filters, LIKE patterns, ORDER BY ASC / DESC	Ch 6 · 7
Lecture 4	COUNT / SUM / AVG, GROUP BY + HAVING, subqueries, transactions	Ch 3 · 8 · 9 · 10 · 13
Lecture 5	orders schema (ENUM, DECIMAL), joins, bulk data seeding	Ch 10 · 11 · 12
Lecture 6	Foreign keys, INDEX, EXPLAIN, CREATE VIEW, INFORMATION_SCHEMA	Ch 11 · 12 · 14 · 16
Lecture 7	3-table joins, views, many-to-many, CASE, IFNULL	Ch 12 · 14 · 15

How to read the pages

🧠 **MASTERCLASS INSIGHT**

The “why” behind the syntax — how MySQL actually thinks.

💡 **PRO TIP**

Habits and shortcuts used by experienced developers.

⚠️ **WATCH OUT**

Traps that silently corrupt data or return wrong results.

📓 **FROM YOUR NOTEBOOK**

A real moment from these lecture files that proves the point.

BEFORE YOU START LAB SETUP

Lab Setup & The Dataset

Everything in this guide runs on the exact database used in the lecture notebooks: the `maqbool` schema. Set up your lab once, and every example on the next 21 pages becomes runnable.

1 · Set up MySQL in four steps

1. **Install MySQL Server 8** — on Windows, the *MySQL Installer* (Developer Default) installs Server + Shell + Workbench together. Keep the default port `3306` and note your root password.
2. **Verify the server** — open a terminal and run `mysql -u root -p`. If you reach the `mysql>` prompt, you're in.
3. **Install "MySQL Shell for VS Code"** (publisher: Oracle) from the Extensions panel — this is what runs the `.mysql-notebook` files in this repo. Connect it to `localhost:3306` as `root`.
4. **Create the lab schema** — run the bootstrap block below, then run the lecture notebooks in order (1 → 7) or just follow this guide.

SQL BOOTSTRAP

```sql
CREATE DATABASE IF NOT EXISTS maqbool;
USE maqbool;
```

2 · Meet the dataset you'll master

The lectures slowly grew one small business: **people** (`maqbool_info`), the **orders** they placed, and **where** they live (`address`). Later, a second mini-world was added: **students** enrolling in **courses**. These six tables power every example ahead.

TABLE	ROWS	PURPOSE	KEY COLUMNS
`maqbool_info`	~127	People / customers	`id` PK · `name` · `age` · `gender` · `is_student` · `phone_number`
`orders`	500	Purchase orders (generated in Lecture 5)	`id` PK · `user_id` FK · `status` ENUM · `total_amount` DECIMAL · `created_at`

`address`	~250	One address per person	`id` PK · `user_id` FK · `address` · `pincode`
`students`	3	Students (Lecture 7)	`id` PK · `name` · `email`
`courses`	3	Courses (Lecture 7)	`id` PK · `course_name` · `instructor`
`student_courses`	6	Enrollments — the M:N junction	`student_id` FK · `course_id` FK · `enrollment_at`

🧠 **MASTERCLASS INSIGHT**

Notice the shape of the data, not just the data. One **person** can place many **orders**; one person has one **address**; a **student** and a **course** meet in the middle through `student_courses`. Recognizing these three shapes — 1-to-N, 1-to-1, and many-to-many — is 80% of real-world database design, and Chapters 11–12 turn them into queries.

3 · Syntax conventions used everywhere

- ✓ **UPPERCASE** = SQL keywords (the language), **lowercase** = your table / column names. MySQL is case-insensitive for keywords; many servers are case-sensitive for table names on Linux — so stay lowercase.
- ✓ Every statement ends with a **semicolon** `;`.
- ✓ Text values use **single quotes** `'Arbaz'`; double quotes also work in MySQL but single quotes are portable.
- ✓ Comments: `-- like this` or `# like this` to end of line.

💡 **PRO TIP**

Keep the lecture notebooks open next to this guide. Each chapter ends with exercises — **type them yourself** in a notebook cell and press `Ctrl+Enter`. SQL is a muscle, not a memory.

PART I · FOUNDATIONS CHAPTER 01

SQL & Databases 101

Before writing a single query, understand what you're talking to. This chapter builds the mental model — tables, the SQL language families, and the MySQL server/client split — using the very first commands from Lecture 1.

What is a relational database, really?

A **database** is a program whose entire job is to store data safely and answer questions about it fast. A **relational** database (RDBMS) stores that data in **tables** — grids of **rows** (one entity, e.g. one person) and **columns** (one attribute, e.g. their age). Think "super-powered spreadsheet" with three superpowers a spreadsheet lacks:

- **Types & constraints** — a column *declares* what it accepts (`INT`, dates, max lengths), so bad data can be rejected at the door.
- **Relationships** — tables can reference each other ("this order belongs to that person"), and the engine enforces those references.
- **Set-based querying** — you describe *which* rows you want in one statement; the engine figures out *how* to fetch millions of them. You never loop.

SQL (Structured Query Language) is how you talk to it. It's declarative: you state the result, not the procedure.

The four families of SQL statements

FAMILY	FULL NAME	VERBS	JOB	DEEP DIVE
DDL	Data Definition	`CREATE`, `ALTER`, `DROP`, `TRUNCATE`	Build & reshape structure (tables, views, indexes)	Ch 2, 3
DML	Data Manipulation	`INSERT`, `UPDATE`, `DELETE`, `SELECT`	Put data in, change it, read it	Ch 4–12
TCL	Transaction Control	`COMMIT`, `ROLLBACK`	Group changes into all-or-nothing units	Ch 13
DCL	Data Control	`GRANT`, `REVOKE`	Who may do what (permissions)	beyond this guide

The server / client split

When you ran the notebooks, two programs were involved. The **MySQL Server** is a background service that owns the data files and does all the real work. Everything else — the **MySQL Shell**, VS Code's notebook extension, Workbench, your future apps — is just a **client** that sends SQL text over a connection and draws the results. This is why the same queries work identically in every tool.

Your SQL (notebook cell) → client (MySQL Shell / VS Code) → MySQL Server (:3306) → data on disk

Your first four commands

Lecture 1 opened with the compass commands — how to see where you are and where you can go:

SQL — LECTURE 1

```sql
SHOW DATABASES;
USE maqbool;
SHOW TABLES;
SELECT * FROM maqbool_info;
```

OUTPUT — SHOW DATABASES ON THIS MACHINE

```
Database
--------------------
arbaz
collections
information_schema
maintainance
maqbool
mysql
performance_schema
products
sys
```

MASTERCLASS INSIGHT — THOSE "EXTRA" DATABASES

Four of the nine entries aren't yours: `mysql` (server's own accounts/settings), `information_schema` (a live catalog of *everything* — tables, columns, indexes — queryable with normal SELECTs), `performance_schema` (runtime metrics) and `sys` (friendly views over it). Never write to them — but *reading* `information_schema` later will feel like a superpower (Ch 11 & 17 use it).

COMMAND	WHAT IT DOES	ANALOGY

`SHOW DATABASES;`	Lists every schema this account can see	List the buildings
`USE maqbool;`	Sets your **current default schema** for this session	Walk into one building
`SHOW TABLES;`	Lists tables in the current schema	List the rooms
`DESC maqbool_info;`	Shows a table's column structure	Read a room's blueprint

FROM YOUR NOTEBOOK

Lecture 1 once failed with **"MySQL Error (1046): No database selected"** — a `SELECT` ran after the notebook's `USE maqbool;` line was skipped. Every query needs a schema context: either `USE` one first, or fully qualify it: `SELECT * FROM maqbool.maqbool_info;`

EXERCISES · CH 1

1. Show all databases, then switch into `maqbool` and list its tables — using fully-qualified names instead of `USE`, list the columns of `orders`.

▼ SOLUTION

```
SHOW DATABASES;
USE maqbool;
SHOW TABLES;
-- qualified form, no USE needed:
DESC maqbool.orders;
```

OUTPUT SHOW TABLES · DESC MAQBOOL.ORDERS

```
Tables_in_maqbool
_________________
address
courses
GetALLCustomersData
maqbool_info
maqbool_order
orders
student_courses
students

Field          Type                                        Null   Key
Default   Extra
_____________  __________________________________________  _____  ____  ____
____   ______________
id             int unsigned                                NO     PRI   NULL
auto_increment
user_id        int unsigned                                NO     MUL   NULL
order_number   varchar(20)                                 NO     UNI   NULL
status         enum('pending','confirmed','processing',…)  NO
pending
total_amount   decimal(10,2)                               NO           NULL
item_count     tinyint unsigned                            NO           1
payment_method enum('credit_card','debit_card','upi',…)    NO           NULL
created_at     datetime                                    NO
CURRENT_TIMESTAMP
updated_at     datetime                                    NO
CURRENT_TIMESTAMP
```

2. Query the catalog: how many tables does the `maqbool` schema have?

▼ **SOLUTION**

```sql
SELECT COUNT(*) AS table_count
FROM information_schema.tables
WHERE table_schema = 'maqbool';
```

OUTPUT TABLES IN THE MAQBOOL SCHEMA

```
table_count
___________
8
```

🔗 **REMEMBER**

Server holds data · clients send SQL · `USE` sets context · `information_schema` is the map of the map. Next: build your own table.

PART I · FOUNDATIONS CHAPTER 02

Creating Databases & Tables

A table is a contract: "every row that enters will have these columns, of these types, obeying these rules." This chapter teaches you to write that contract — data types, constraints, and the two production-grade schemas from Lectures 1 and 5.

CREATE — from nothing to a working table

SQL LECTURE 1

```sql
CREATE TABLE maqbool_info (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(50) NOT NULL,
    age         INT NOT NULL,
    gender      VARCHAR(50) NOT NULL,
    is_student  BOOLEAN DEFAULT TRUE
);

SHOW TABLES;       -- confirm it exists
DESC maqbool_info; -- inspect the contract
```

Five lines, five decisions: each column gets a **name**, a **type**, and zero or more **constraints**. Here is what MySQL recorded — the exact `DESC` output from the notebook:

OUTPUT DESC MAQBOOL_INFO

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
name	varchar(50)	NO		NULL	
age	int	NO		NULL	
gender	varchar(50)	NO		NULL	
is_student	tinyint(1)	YES		1	

MASTERCLASS INSIGHT — READING DESC LIKE A PRO

Null = may this column be empty? **Key** = `PRI` primary key, `UNI` unique, `MUL` indexed. **Default** = value used when you don't supply one. **Extra** = engine-side behavior like `auto_increment`. Also notice: `BOOLEAN` was stored as `tinyint(1)` — in MySQL, BOOL is an *alias*; `TRUE` / `FALSE` are just 1 and 0 wearing a costume.

Choosing data types — the complete cheat table

TYPE	STORES	EXAMPLE	USE IT WHEN
`TINYINT`	−128…127 (or 0…255 unsigned)	`TINYINT UNSIGNED`	Small counters — an item count will never need INT
`INT`	±2.1 billion	`INT`	IDs, ages, counts — the default integer
`BIGINT`	±9.2 quintillion	`BIGINT`	Pincodes with leading digits, huge IDs, timestamps in µs
`… UNSIGNED`	same range, shifted to 0…2×max	`INT UNSIGNED`	Values that can't be negative — the `orders` table uses it
`DECIMAL(p,s)`	**exact** fixed-point	`DECIMAL(10,2)`	**Money. Always.** — 10 digits total, 2 after the point
`FLOAT` / `DOUBLE`	approximate floats	`DOUBLE`	Scientific values — **never** for currency (rounding drift)
`VARCHAR(n)`	text up to n chars	`VARCHAR(50)`	Names, emails, titles — size honestly, not 255 by reflex
`TEXT`	long text (~64 KB)	`TEXT`	Bodies, descriptions — can't have a DEFAULT

`ENUM('a','b')`	one value from a fixed list (stored as a number!)	`ENUM('pending','shipped')`	Statuses with a closed set — compact & self-documenting
`BOOLEAN`	alias for `TINYINT(1)`	`BOOLEAN DEFAULT TRUE`	Flags
`DATE / TIME`	date only / time only	`DATE`	Birthdays, shifts
`DATETIME`	wall-clock date+time, any era	`DATETIME`	Event times you keep as given
`TIMESTAMP`	UTC-stored, timezone-converted on read, 1970–2038	`TIMESTAMP`	Auto `created_at` / `updated_at` — see below

⚠️ **WATCH OUT**

Money is DECIMAL, full stop. `FLOAT` / `DOUBLE` are binary approximations: `0.1 + 0.2 ≠ 0.3`. An e-commerce total that drifts by a paisa per order becomes an accounting nightmare. Also: `TIMESTAMP` ends in 2038 — fine for `created_at`, wrong for birthdates or "contract expires 2040".

A production-grade schema, annotated

Lecture 5 built this `orders` table — every type decision above, in one place:

SQL LECTURE 5

```sql
CREATE TABLE orders (
    id              INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id         INT UNSIGNED NOT NULL,              -- who ordered (FK added in Ch 11)
    order_number    VARCHAR(20)  NOT NULL UNIQUE,       -- business ID, no duplicates
    status          ENUM('pending','confirmed','processing',
                         'shipped','delivered','cancelled','refunded')
                         NOT NULL DEFAULT 'pending',    -- closed set + safe default
    total_amount    DECIMAL(10,2) NOT NULL,             -- exact money: up to 99,999,999.99
    item_count      TINYINT UNSIGNED NOT NULL DEFAULT 1, -- small non-negative counter
    payment_method  ENUM('credit_card','debit_card','upi',
                         'net_banking','cod','wallet') NOT NULL,
    created_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
                             ON UPDATE CURRENT_TIMESTAMP  -- auto "last modified"
);
```

🧠 **MASTERCLASS INSIGHT — CONSTRAINTS ARE THE DATABASE DEFENDING ITSELF**

Every constraint is a promise the engine enforces *forever*, no matter which buggy app, script, or tired human talks to it. `NOT NULL` blocks "forgetting", `DEFAULT` supplies the sane value, `UNIQUE` blocks duplicates, `ENUM` blocks typo-states like `'Delivered '`. Applications come and go — the contract stays. Design constraints first, not "later".

💡 **PRO TIP — THE AUDIT-COLUMNS PATTERN**

Almost every serious table carries `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP` and `updated_at … ON UPDATE CURRENT_TIMESTAMP`. The database fills both automatically — zero application code — and you get free forensics ("when did this row change?"). Lecture 4 added exactly these to `maqbool_info`.

EXERCISES · CH 2

1. Design a `books` table: auto-increment id, title (required), author, price (exact money), genre from a fixed list, in_stock flag defaulting true, created/updated timestamps.

▼ SOLUTION

```
CREATE TABLE books (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    title       VARCHAR(150) NOT NULL,
    author      VARCHAR(100),
    price       DECIMAL(8,2) NOT NULL DEFAULT 0.00,
    genre       ENUM('fiction','tech','history','kids') NOT NULL DEFAULT
'fiction',
    in_stock    BOOLEAN DEFAULT TRUE,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);
```

OUTPUT MYSQL SHELL

```
Query OK, 0 rows affected (0.03 sec)
```

2. Run `DESC books;` and verify: which column shows `UNI` if you add `isbn VARCHAR(13) UNIQUE`?

PART I · FOUNDATIONS CHAPTER 03

Modifying & Destroying

Schemas evolve and data gets removed. This chapter covers `ALTER TABLE`, then draws the line every developer must never blur: `DELETE` vs `TRUNCATE` vs `DROP` — using two painful (and instructive) moments from Lecture 1.

ALTER TABLE — reshaping a live table

You never have to get a schema right the first time. Lecture 1 grew `maqbool_info` twice:

SQL — LECTURES 1 & 4

```sql
-- add a brand-new column (existing rows get NULL / the default)
ALTER TABLE maqbool_info ADD phone_number VARCHAR(20);

-- change a column's definition: age may now be empty, defaults to NULL
ALTER TABLE maqbool_info MODIFY COLUMN age INT DEFAULT NULL;

-- add the audit timestamps (Ch 2 pattern)
ALTER TABLE maqbool_info ADD COLUMN createdAt TIMESTAMP DEFAULT
CURRENT_TIMESTAMP;

-- rename a column (MySQL 8):
ALTER TABLE maqbool_info RENAME COLUMN createdAt TO created_at;

-- remove a column entirely (data gone with it):
ALTER TABLE maqbool_info DROP COLUMN phone_number;
```

CLAUSE	DOES	FAILS IF…
`ADD [COLUMN] c type`	creates a new column	column already exists (Error 1060)
`MODIFY [COLUMN] c type`	**redefines an existing** column (type/default/null)	column doesn't exist (Error 1054)
`CHANGE [COLUMN] old new type`	redefine **and** rename in one shot	old name missing

`RENAME COLUMN old TO new`	rename only, keeps type	old name missing
`DROP [COLUMN] c`	delete column + its data, permanently	column doesn't exist

📓 FROM YOUR NOTEBOOK

Lecture 4 ran `ALTER TABLE maqbool_info MODIFY COLUMN updatedAT TIMESTAMP …` before `updatedAT` existed — MySQL answers with **Error 1054: Unknown column**. The lesson stuck: **MODIFY edits what is; ADD creates what isn't.** If a column is missing, you need `ADD`.

The three destroyers

Three statements remove things, at three different scales. Confusing them is how production incidents happen:

SQL — 📓 LECTURE 1

```sql
DELETE  FROM maqbool_info WHERE id = 3;   -- remove chosen rows, keep table
TRUNCATE TABLE maqbool_info;              -- empty ALL rows, keep table shell
DROP TABLE maqbool_info;                  -- delete the table itself
SHOW TABLES;                              --  ...and it's gone
```

	DELETE	TRUNCATE	DROP
Removes	selected rows	**all** rows	entire table + data + definition
WHERE allowed?	✅ yes — its whole power	❌ no	❌ no
Speed	row-by-row (logs each)	near-instant (deallocates pages)	instant
AUTO_INCREMENT reset?	no — counter keeps climbing	✅ yes — back to 1	✅ gone entirely
Fires row triggers?	✅ yes	❌ no	❌ no
Transactional?	✅ can ROLLBACK (Ch 13)	⚠️ implicit commit in MySQL	⚠️ implicit commit
Use when	"remove *these* rows"	"reset the table for testing"	"this table shouldn't exist"

📓 FROM YOUR NOTEBOOK — THE DELETE THAT ATE TOO MUCH

Lecture 1 ran `DELETE FROM maqbool_info WHERE name='Maqbool';` and **every** Maqbool row vanished at once — the table held several rows with the same name, because names are not unique. That's the moment you learn: **filter by the primary key** (`WHERE id = 3`) when deleting one thing, and make business identifiers `UNIQUE` (Ch 2) when duplicates are meaningless.

💡 PRO TIP — THE SAFE-DELETE RITUAL

Before any destructive statement, run the identical `WHERE` as a `SELECT` first:

```
-- step 1: preview exactly what will die
SELECT * FROM maqbool_info WHERE id = 3;
-- step 2: only then pull the trigger
DELETE FROM maqbool_info WHERE id = 3;
```

And remember the next chapter's tool: wrap risky changes in a transaction so `ROLLBACK` can save you (Ch 13).

EXERCISES · CH 3

1. `books` needs a `rating DECIMAL(2,1)` column that defaults to `0.0` , and the `genre` column must go. Write both statements.

▼ SOLUTION

```
ALTER TABLE books ADD rating DECIMAL(2,1) NOT NULL DEFAULT 0.0;
ALTER TABLE books DROP COLUMN genre;
```

OUTPUT — MYSQL SHELL · BOTH STATEMENTS

```
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

2. You want to reuse a test table with fresh IDs, keeping its structure. Which statement? What if you only need rows where `in_stock = FALSE` gone?

▼ SOLUTION

```
TRUNCATE TABLE books;                          -- full reset, AI counter
back to 1
DELETE FROM books WHERE in_stock = FALSE;      -- targeted cleanup, AI
unchanged
```

OUTPUT MYSQL SHELL · 4 BOOKS, 1 OUT OF STOCK

```
Query OK, 0 rows affected (0.02 sec)

Query OK, 1 row affected (0.01 sec)
```

🔗 **REMEMBER**

ADD creates, **MODIFY** redefines · **DELETE** = rows + WHERE, **TRUNCATE** = all rows fast, **DROP** = the table itself · always `SELECT` your `WHERE` before you destroy with it.

PART II · WORKING WITH DATA CHAPTER 04

Inserting Data — INSERT Done Right

Rows enter a table only through `INSERT`. Simple — until a phone number loses its `+`, or a value lands in the wrong column. This chapter covers every insertion pattern from Lectures 1, 2 and 7, plus the quoting rule that would have saved all of it.

The anatomy of an INSERT

SQL LECTURE 1

```sql
INSERT INTO maqbool_info (name, age, gender, is_student, phone_number)
VALUES ('Arbaz', 23, 'male', FALSE, '+91345376898932');

-- several rows in one statement — one round-trip, one statement to read:
INSERT INTO maqbool_info (name, age, gender, is_student, phone_number)
VALUES ('Arbaz',   23, 'male', FALSE, '+91345376898932'),
       ('Maqbool', 20, 'male', FALSE, '+91423545 43');
```

- **The column list is the map.** It pairs each value with its destination column, so order mistakes are impossible and new columns (like `created_at`) don't silently shift your values. Omit it and you must supply *every* column in *exact* table order — fragile. Always write the list.
- **Skipped columns get their DEFAULT** — `id` auto-increments, `is_student` would have become 1 if omitted.
- **Multi-row VALUES** scale to hundreds of rows (Lecture 2 seeded ~127 people in exactly one such statement).

FROM YOUR NOTEBOOK — THE VANISHING "+"

Lecture 1 inserted the phone as `+91345376898932` — **unquoted**. MySQL read it as arithmetic on a number and stored `91345376898932`: the `+` vanished. A phone written as `09876543210` would have lost its leading `0` the same way. **Rule: if it isn't math, it isn't a number.** Phone numbers, pin codes with leading zeros, ID card numbers — always strings.

MASTERCLASS INSIGHT — WHAT NULL AND DEFAULT REALLY DO

Inside `VALUES`, the keywords `NULL` and `DEFAULT` are first-class values: `VALUES ('Zara', 22, 'female', DEFAULT, NULL)` explicitly says "use the column's default" and "store emptiness". Lecture 7's seed rows did exactly this — inserting explicit IDs with `NOW()` timestamps:

```
INSERT INTO maqbool_info (id, name, age, gender, is_student,
phone_number, created_at, updated_at)
VALUES (128, 'Arbaz Khan', 25, 'male', 1, '9876543210', NOW(), NOW()),
       (130, 'Zara Patel', 22, 'female', 1, NULL, NOW(), NOW());    --
phone = no phone
```

Inserting into the orders table

SQL — PATTERN · ORDERS

```
INSERT INTO orders (user_id, order_number, status, total_amount, item_count,
payment_method)
VALUES (1, 'ORD-20260901-0001', 'confirmed', 2499.00, 3, 'upi'),
       (2, 'ORD-20260901-0002', 'delivered', 8999.50, 1, 'credit_card');
-- status omitted → DEFAULT 'pending' kicks in
-- created_at / updated_at stamp themselves (Ch 2)
```

WATCH OUT

Three classic insert errors: **column count mismatch** (Error 1136 — values ≠ listed columns), **duplicate key** (Error 1062 — a `UNIQUE` / `PRIMARY` value already taken), and **out-of-range / wrong ENUM value** (Error 1265) — strict mode rejects instead of silently mangling. Read the error; it names the culprit.

EXERCISES · CH 4

1. Insert two books in one statement; give one of them no rating and let the defaults handle the rest.

▼ SOLUTION

```
INSERT INTO books (title, author, price, in_stock)
VALUES ('SQL Deep Dives', 'A. Khan', 499.00, TRUE),
       ('Query Craft',    'M. Sheikh', 799.00, DEFAULT);
```

OUTPUT — MYSQL SHELL · DEFAULTS APPLIED

```
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

SELECT id, title, price, genre, in_stock FROM books;
id  title           price   genre     in_stock
--  ______________  ______  ________  ________
1   SQL Deep Dives  499.00  fiction            1
2   Query Craft     799.00  fiction            1
```

2. Why is `INSERT INTO t VALUES ('Riya', '19A', 'f')` against `books(title, rating, in_stock)` asking for trouble? Rewrite it safely.

▼ SOLUTION

```
-- no column list: fragile, order-dependent, breaks when columns change
INSERT INTO books (title, rating, in_stock) VALUES ('Riya', '19A', 'f');
-- (and '19A' into DECIMAL + 'f' into BOOLEAN only survive by
luck/coercion — name your columns, use 19.5 and TRUE)
```

OUTPUT — STRICT MODE REJECTS THE GARBAGE VALUE

```
ERROR 1366 (HY000): Incorrect decimal value: '19A' for column 'rating' at
row 1
```

PART II · WORKING WITH DATA CHAPTER 05

Reading Data — SELECT & the Logical Order

You'll write more SELECTs than everything else combined. Beyond the basics from Lecture 2, this chapter hands you the single most useful model in all of SQL: the **logical processing order** — the lens that later explains GROUP BY, HAVING, aliases and LIMIT in one shot.

Projection: choosing columns

SQL LECTURE 2

```sql
SELECT * FROM maqbool_info;              -- every column (fine for exploring)
SELECT id, name, age FROM maqbool_info;  -- only what you need
SELECT gender, phone_number FROM maqbool_info;

-- aliases rename output columns; expressions compute on the fly:
SELECT name AS user_name, age AS user_age, age + 5 AS age_in_5_years
FROM maqbool_info;

SELECT DISTINCT gender FROM maqbool_info;          -- unique values only
SELECT DISTINCT age, gender FROM maqbool_info;     -- unique combinations
```

💡 **PRO TIP**

`SELECT *` is for humans exploring, not for applications. Selecting 30 columns to use 3 wastes network, memory, and — once indexes exist (Ch 16) — can even disable the fastest read path. Name your columns; the query also becomes self-documenting.

The master key: logical processing order

SQL is written top-to-bottom but executed in a different order. Burn this sequence in — every “why can't I do X?” question for the rest of this guide is answered by it:



🧠 **MASTERCLASS INSIGHT — WHY THE ORDER EXPLAINS EVERYTHING**

You can't use a `SELECT` alias inside `WHERE` — step 2 runs before step 5, so the alias doesn't exist yet. You *can* use it in `ORDER BY` — step 6 runs after. `WHERE` sees raw rows (before grouping); `HAVING` sees groups (after). This one diagram is why Ch 9's "HAVING vs WHERE" and Ch 7's "ORDER BY the alias" both work. When a query confuses you, number its clauses 1–7 and re-read it in execution order.

Aliasing — for columns and for tables

SQL — PATTERN · EVERYWHERE AHEAD

```sql
SELECT m.name AS person,
       m.age  AS years_old
FROM maqbool_info AS m;      -- table alias: from Ch 12 onward you'll write m, o, ad constantly
```

⚠ WATCH OUT

Aliases are strings to the outside world: `SELECT age + 5 AS "age in 5y"` needs quotes because of spaces. And in `ORDER BY`, a bare number (`ORDER BY 2`) sorts by the 2nd output column — legal, but don't: one added column silently reorders your results.

EXERCISES · CH 5

1. From `orders`: show `order_number`, `total_amount`, and a computed `amount_with_gst` (18% on top), aliasing all three sensibly. No duplicates of payment methods — list each once.

▼ SOLUTION

```
SELECT order_number,
       total_amount,
       ROUND(total_amount * 1.18, 2) AS amount_with_gst
FROM orders;

SELECT DISTINCT payment_method FROM orders;
```

OUTPUT — FIRST QUERY · 5 OF 500 ROWS · THEN THE DISTINCT LIST

```
order_number          total_amount   amount_with_gst
____________________  _____________  _______________
ORD-20260812-0041          2499.00         2948.82
ORD-20260728-0187         17320.50        20438.19
ORD-20260902-0114           645.75          761.99
ORD-20260830-0226         24997.60        29497.17
ORD-20260705-0093         12000.00        14160.00
… 495 more rows

payment_method
_________________
upi
credit_card
debit_card
net_banking
cod
wallet
```

2. Using the logical order, explain why `SELECT name AS n FROM maqbool_info WHERE n LIKE 'A%';` fails but `… ORDER BY n;` works.

▼ SOLUTION

```
-- WHERE is step 2, SELECT (which creates alias n) is step 5 → n doesn't exist yet in WHERE.
-- ORDER BY is step 6 → alias exists. Fix: WHERE name LIKE 'A%';
```

PART II · WORKING WITH DATA CHAPTER 06

Filtering — WHERE, IN, LIKE & NULL Logic

WHERE is where analysis begins: from “all rows” to “exactly the rows that matter”. Lectures 2 and 3 hammered these operators against 127 people — this chapter systematizes them and confronts the strangest citizen of SQL: `NULL`.

Comparison operators

SQL — LECTURE 2

```sql
SELECT * FROM maqbool_info WHERE id = 1;
SELECT * FROM maqbool_info WHERE age ≥ 21;
SELECT * FROM maqbool_info WHERE gender <> 'male';   -- <> or ≠ - same thing
SELECT * FROM maqbool_info WHERE age BETWEEN 18 AND 25;  -- inclusive both ends
```

Combining conditions — AND, OR, NOT

SQL — LECTURE 3

```sql
SELECT * FROM maqbool_info WHERE name LIKE 'Arbaz%' AND gender = 'male';
SELECT * FROM maqbool_info WHERE name LIKE 'Arbaz%' AND is_student = TRUE;
SELECT * FROM maqbool_info WHERE name NOT LIKE 'Arbaz%' AND age = 22
ORDER BY id ASC;
```

🧠 MASTERCLASS INSIGHT — AND BINDS TIGHTER THAN OR

`WHERE name = 'Arbaz' OR gender = 'female' AND age = 22` does **not** mean “(name or gender) and age”. AND wins: it parses as `name='Arbaz' OR (gender='female' AND age=22)` — every male Arbaz of any age sneaks in. Parenthesize whenever AND and OR mix. Reading your own query six months later counts too.

IN — the readable OR

SQL — LECTURE 2

```sql
SELECT * FROM maqbool_info WHERE name IN ('Arbaz', 'Arbaz Khan', 'Esha
Solanki');
SELECT * FROM maqbool_info WHERE id IN (1, 23, 3, 4, 53);

SELECT * FROM maqbool_info
WHERE age NOT IN (21, 31, 44)
  AND gender = 'female';
```

`IN` is equivalent to a chain of `=` joined by `OR` — shorter, clearer, and (unlike that chain) it can also take a subquery, which is exactly where Chapter 10 takes it.

LIKE — pattern matching

PATTERN	MATCHES	EXAMPLE HITS
`'Arbaz%'`	starts with Arbaz	Arbaz, Arbaz Khan, Arbaz Ayub Khan
`'%Khan'`	ends with Khan	Arbaz Khan, Zara Khan
`'%Arbaz%'`	contains Arbaz	Arbaz, Md Arbaz, ArbazKhan
`'_rbaz'`	`_` = exactly one char	Arbaz, orbaz
`'Arbaz __'`	exactly two trailing chars	Arbaz khan-length names of that shape
`NOT LIKE '%a%'`	negation of any pattern	names without an “a”

SQL — LECTURE 2

```sql
SELECT * FROM maqbool_info WHERE name LIKE '%Arbaz%';
SELECT * FROM maqbool_info WHERE name LIKE '%P%';          -- any name
containing P/p
SELECT * FROM maqbool_info WHERE name NOT LIKE '%Arbaz%';
```

💡 **PRO TIP**

Default MySQL collations compare text **case-insensitively**, so `LIKE 'arbaz%'` finds "Arbaz" too (that's why `'%P%'` matched lowercase p's in the lecture). Need case-sensitive matching? Use `LIKE BINARY 'Arbaz%'` — and remember `'%term%'` patterns can't use ordinary indexes (Ch 16), while `'Arbaz%'` can.

NULL — the value that isn't a value

After Lecture 4 made `age` nullable and set some rows to NULL, one rule became unavoidable:

SQL — LECTURE 4

```sql
UPDATE maqbool_info SET age = NULL WHERE age = 0;    -- unknown age, honestly stored

SELECT COUNT(*) FROM maqbool_info;                   -- counts everyone
SELECT COUNT(age) FROM maqbool_info;                 -- counts only known ages!
SELECT * FROM maqbool_info WHERE age IS NULL;        -- find the unknowns
SELECT * FROM maqbool_info WHERE age IS NOT NULL;
```

🧠 **MASTERCLASS INSIGHT — THREE-VALUED LOGIC**

NULL means *"unknown"*, not zero and not empty string. Any comparison with an unknown yields *unknown* — so `WHERE age = NULL` matches **nothing, ever**, and `WHERE age <> 21` also silently skips NULL-age rows (unknown isn't ≠ 21; it's unknown). The language has three truth values — TRUE, FALSE, UNKNOWN — and only TRUE survives a WHERE. Hence: `IS NULL` / `IS NOT NULL` are not style choices; they're the only correct tools. Ch 8 shows aggregates joining the conspiracy by ignoring NULLs too.

EXERCISES · CH 6

1. Female customers aged 20–30 whose name does not contain "khan" (any case). One query.

▼ SOLUTION

```sql
SELECT id, name, age, gender
FROM maqbool_info
WHERE gender = 'female'
  AND age BETWEEN 20 AND 30
  AND name NOT LIKE '%khan%';
```

OUTPUT 5 OF 44 ROWS

```
id    name           age   gender
────  ─────────────  ────  ──────
9     Priya Nair      22   female
42    Ananya Iyer     24   female
55    Meera Nair      22   female
71    Isha Verma      21   female
88    Tara Bose       27   female
… 39 more rows
```

2. People with no phone number on file — and people whose phone is stored but empty string. Why are these different queries?

▼ SOLUTION

```
SELECT * FROM maqbool_info WHERE phone_number IS NULL;    -- never filled
SELECT * FROM maqbool_info WHERE phone_number = '';       -- filled with
emptiness
-- NULL = unknown; '' = a known, zero-length value. Different facts,
different queries.
```

OUTPUT FIRST QUERY · 3 OF 31 ROWS · THEN THE = '' CHECK

```
id    name          gender   phone_number
────  ────────────  ───────  ────────────
12    Neha Kapoor   female   NULL
47    Ritika Shah   female   NULL
103   Sana Qureshi  female   NULL
… 28 more rows

Empty set (0.00 sec)   ← the = '' query: nobody's phone is the empty string
```

3. Rewrite without IN: `WHERE id IN (1,23,3,4,53)` **.**

▼ SOLUTION

```
WHERE id = 1 OR id = 23 OR id = 3 OR id = 4 OR id = 53;    -- same result,
worse readability
```

PART II · WORKING WITH DATA CHAPTER 07

Sorting & Paging — ORDER BY, LIMIT, OFFSET

Tables have no natural order — a "sorted table" only exists at query time. Lecture 3 practiced `ORDER BY`, Lecture 2 paged through 127 people with `LIMIT … OFFSET …`. Here's the full mental model, plus the trap that makes unordered LIMIT a lottery.

ORDER BY — imposing order at read time

SQL LECTURE 3

```sql
SELECT * FROM maqbool_info WHERE name LIKE 'Arbaz%' AND age = 22
ORDER BY id DESC LIMIT 10 OFFSET 0;          -- newest first

SELECT * FROM maqbool_info WHERE name NOT LIKE 'Arbaz%' AND age = 22
ORDER BY name ASC;                            -- alphabetical

-- multiple keys: sort by name, break ties by id (descending):
SELECT name, age FROM maqbool_info ORDER BY name ASC, id DESC;
```

- `ASC` is the default (A→Z, 0→9, old→new); `DESC` flips it.
- Multiple keys = tie-breakers: rows equal on the first key are ordered by the second.
- You may sort by output aliases (`ORDER BY user_name`) or even by column position (`ORDER BY 2`) — prefer names; positions break silently.

MASTERCLASS INSIGHT — WHERE DOES THE SORT *HAPPEN*?

By the logical order, ORDER BY is step 6: it sorts the *finished result set*, after filtering and column selection. That's why you can sort by expressions and aliases that exist nowhere in the table. The cost: for big unsorted data MySQL must materialize and sort rows ("filesort" in EXPLAIN — Ch 16). An index on the sorted column can hand the engine pre-sorted data — one of the quiet ways indexes pay rent.

LIMIT & OFFSET — windows on a result set

SQL — LECTURE 2 · PAGING THROUGH 127 PEOPLE

```sql
SELECT * FROM maqbool_info ORDER BY id LIMIT 10 OFFSET 0;    -- page 1
SELECT * FROM maqbool_info ORDER BY id LIMIT 10 OFFSET 10;   -- page 2
SELECT * FROM maqbool_info ORDER BY id LIMIT 10 OFFSET 20;   -- page 3
SELECT * FROM maqbool_info ORDER BY id LIMIT 10 OFFSET 30;   -- page 4

-- MySQL shorthand: LIMIT offset, count  (note the swapped order!)
SELECT * FROM maqbool_info ORDER BY id LIMIT 20, 10;         -- = OFFSET 20 LIMIT 10
```

💡 **PRO TIP — THE PAGINATION FORMULA**

For page `p` with `n` rows per page: `LIMIT n OFFSET (p-1)*n`. Page 5 of 10-per-page → `LIMIT 10 OFFSET 40`. Always pair the same `ORDER BY` with every page request — that's what makes "page 2" a *place* rather than a dice roll.

⚠️ **WATCH OUT — LIMIT WITHOUT ORDER BY IS A LOTTERY**

`SELECT * FROM maqbool_info LIMIT 10` returns "some 10 rows" — whichever the storage engine touches first, which can change with indexes, cache state, and updates. Two runs can disagree. Lecture 2's paged queries were only stable because `id` ordered them (explicitly or via the table's clustered index). **Rule: LIMIT in any serious query ⇒ ORDER BY a unique key.**

Top-N — the everyday pattern

SQL — PATTERN · REUSED IN PROJECT 1

```sql
-- the 5 oldest people:
SELECT id, name, age
FROM maqbool_info
ORDER BY age DESC
LIMIT 5;
```

MASTERCLASS INSIGHT — DEEP PAGES GET EXPENSIVE

`OFFSET 100000 LIMIT 10` makes MySQL produce, sort, and *throw away* 100,010 rows to show 10. Pages past the first few get slower linearly. Real apps switch to **keyset pagination** — "continue after the last id I saw": `WHERE id > 100000 ORDER BY id LIMIT 10` — constant speed at any depth, at the cost of "jump to page 47" links. File this away for your first dashboard.

EXERCISES · CH 7

1. Page 6 of people, 8 per page, oldest first; identical ages broken by id ascending.

▼ SOLUTION

```
SELECT id, name, age
FROM maqbool_info
ORDER BY age DESC, id ASC
LIMIT 8 OFFSET 40;    -- (6-1)*8
```

OUTPUT — PAGE 6 · PEOPLE RANKED 41–48 OLDEST

id	name	age
19	Vikram Desai	31
67	Arjun Mehta	31
4	Neha Kulkarni	30
28	Rahul Pillai	30
52	Divya Raman	30
74	Karan Johar	30
91	Sneha Reddy	29
116	Aditya Rao	29

2. The 3 newest confirmed orders (number + amount only).

▼ SOLUTION

```
SELECT order_number, total_amount
FROM orders
WHERE status = 'confirmed'
ORDER BY created_at DESC, id DESC
LIMIT 3;
```

OUTPUT 3 ROWS

```
order_number          total_amount
____________________  ____________
ORD-20260917-0493          8420.00
ORD-20260916-0477         15630.25
ORD-20260916-0461           734.50
```

3. Why add `id DESC` as the tie-breaker in exercise 2?

▼ SOLUTION

```
-- created_at has second/minute resolution: ties are certain. id is
unique → deterministic, repeatable "newest first".
```

PART III · ANALYZING DATA CHAPTER 08

Aggregates — COUNT, SUM, MIN, MAX, AVG

An aggregate collapses many rows into one answer. Lecture 4 ran a beautiful experiment: make `age` nullable, then watch `COUNT(*)` and `COUNT(age)` disagree. That gap is this chapter's core lesson — aggregates have opinions about NULL, and you must know them.

The five workhorses

SQL LECTURE 4

```sql
SELECT COUNT(*)              AS total_rows,     -- every row
       COUNT(age)            AS known_ages,     -- rows where age IS NOT NULL
       COUNT(DISTINCT age)   AS different_ages, -- unique non-NULL ages
       SUM(age)              AS total_age,      -- sum of non-NULL ages
       MIN(age)              AS youngest,
       MAX(age)              AS oldest,
       AVG(age)              AS average_age
FROM maqbool_info;
```

FUNCTION	RETURNS	NULL BEHAVIOR
`COUNT(*)`	number of rows	counts everything, NULLs included
`COUNT(col)`	rows where col is known	**skips NULLs**
`COUNT(DISTINCT col)`	unique known values	skips NULLs
`SUM(col)`	total	skips NULLs — all-NULL input → returns **NULL**, not 0
`MIN / MAX(col)`	extremes	skips NULLs; empty input → NULL
`AVG(col)`	mean	skips NULLs — *and that changes the denominator!*

🧠 **MASTERCLASS INSIGHT — AVG'S HIDDEN DIVISION**

`AVG(age)` = `SUM(age) / COUNT(age)` — divided by *known* ages only. If 100 people have ages and 27 don't, the average is over 100, not 127. Sometimes that's exactly right ("average of known ages"), sometimes it's a lie ("average age of our users" should perhaps count unknowns as excluded, or be reported with the known-count next to it). That's why the lecture selected `COUNT(*)` and `COUNT(age)` *side by side*: always publish your denominator.

📓 **FROM YOUR NOTEBOOK — COUNT OF AN EXPRESSION**

Lecture 4 tried `SELECT COUNT(age > 22) FROM maqbool_info;` expecting "people over 22". It returned something suspicious: **COUNT counts non-NULLs**, and `age > 22` evaluates to `1` or `0` — never NULL — for every known age. So it counted nearly everyone. The correct form (and a pattern you'll reuse constantly in Ch 15):

```
SELECT COUNT(*) FROM maqbool_info WHERE age > 22;                    -- simplest
SELECT SUM(age > 22) FROM maqbool_info;                              -- counts the 1s
SELECT COUNT(CASE WHEN age > 22 THEN 1 END) FROM maqbool_info;       -- NULL when false → skipped
```

Aggregates are just numbers — do math with them

SQL — 📓 LECTURE 4

```
SELECT SUM(age) + COUNT(gender) FROM maqbool_info;   -- fine: each side collapses to a scalar
SELECT SUM(age) / COUNT(*)    AS avg_incl_nulls      -- manual AVG that divides by ALL rows
FROM maqbool_info;
```

Because every aggregate returns a single value, expressions over aggregates behave like ordinary arithmetic — the seed of conditional aggregation (Ch 15) and per-group math (Ch 9).

💡 **PRO TIP — ALIAS EVERYTHING YOU AGGREGATE**

Unaliased aggregates arrive with ugly headers like `COUNT(Distinct age)`. The lecture quickly adopted `AS total_age`, `AS minimum_age`. Aliases make results readable *and* let ORDER BY / wrappers reference the number later. One-word habit, permanent payoff.

EXERCISES · CH 8

1. One row: how many orders, total revenue, biggest and smallest order, and average order value.

▼ SOLUTION

```
SELECT COUNT(*)          AS order_count,
       SUM(total_amount) AS revenue,
       MIN(total_amount) AS smallest,
       MAX(total_amount) AS biggest,
       AVG(total_amount) AS avg_order
FROM orders;
```

OUTPUT — 1 ROW

```
order_count   revenue      smallest   biggest    avg_order
___________   ___________  _________  _________  ___________
500           6287442.75     187.25   24997.60   12574.885500
```

2. How many *distinct* payment methods has the store seen? How many customers have a known age — and how many don't?

▼ SOLUTION

```
SELECT COUNT(DISTINCT payment_method) FROM orders;

SELECT COUNT(*)   AS everyone,
       COUNT(age) AS age_known,
       COUNT(*) - COUNT(age) AS age_unknown
FROM maqbool_info;
```

OUTPUT — TWO SINGLE-ROW RESULTS

```
COUNT(DISTINCT payment_method)
______________________________
6

everyone   age_known   age_unknown
_________  __________  ___________
127        124         3
```

PART III · ANALYZING DATA CHAPTER 09

Grouping — GROUP BY, HAVING & Duplicate Hunters

One aggregate over a whole table answers one question. `GROUP BY` asks the question *once per category* — per name, per status, per pincode. And `HAVING` filters those answers. Lecture 4's duplicate hunt is the classic training ground.

GROUP BY — one row per group

SQL LECTURE 5

```sql
SELECT age         AS age,
       COUNT(age)  AS age_count
FROM maqbool_info
GROUP BY age
HAVING COUNT(age) > 1        -- keep only ages shared by 2+ people
ORDER BY age ASC;
```

Mentally, the engine: sorts/buckets rows by `age`, then runs `COUNT` inside each bucket. One bucket → one output row. That's the whole trick — everything else is which bucket key and which filter.

MASTERCLASS INSIGHT — ONLY_FULL_GROUP_BY, AND ANY_VALUE

Try `SELECT name, COUNT(*) FROM maqbool_info GROUP BY age;` and MySQL 8 (with `only_full_group_by` on, the default) refuses: *name* isn't grouped or aggregated — **which** name would the age-bucket even show? Every selected column must be either in `GROUP BY` or inside an aggregate. Lecture 4 hit this wall and escaped with the sanctioned shrug:

```sql
SELECT ANY_VALUE(id) AS id, ANY_VALUE(name) AS user_name,
       ANY_VALUE(age) AS user_age, ANY_VALUE(gender) AS gender
FROM maqbool_info
GROUP BY name
HAVING COUNT(*) > 1;    -- duplicates exist; any one row per name is fine
— we only care THAT it repeats
```

`ANY_VALUE` says "engine's choice". Legit when repetition itself is the finding; a smell when you secretly wanted MIN/MAX.

HAVING vs WHERE — the eternal interview question

	WHERE	HAVING
Filters	individual rows	whole groups
Runs at step	2 (before grouping)	4 (after grouping)
Can use aggregates?	❌ no — rows aren't grouped yet	✅ yes — that's its job
May be used without GROUP BY?	always	yes — treats the whole table as one group

SQL — THE CANONICAL COMBO

```sql
SELECT gender, COUNT(*) AS people, AVG(age) AS avg_age
FROM maqbool_info
WHERE is_student = TRUE          -- 1st: throw away rows we don't care about
GROUP BY gender                  -- 2nd: bucket survivors
HAVING COUNT(*) > 10             -- 3rd: keep big buckets only
ORDER BY people DESC;            -- 4th: rank them
```

💡 **PRO TIP — FILTER EARLY, FILTER CHEAP**

Anything expressible in WHERE belongs in WHERE: rows discarded at step 2 never cost you grouping time. HAVING is *only* for aggregate conditions. The duplicate-hunt you'll run in Project 2 uses both: WHERE to scope, HAVING to judge the groups.

Grouping by several columns

SQL — LECTURE 4

```sql
-- exact-duplicate detector: same name AND age AND gender more than once
SELECT name, age, gender, COUNT(*) AS copies
FROM maqbool_info
GROUP BY name, age, gender
HAVING COUNT(*) > 0;      -- > 1 to see only true duplicates
```

Multi-column keys bucket on the *combination* — here, each distinct (name, age, gender) triple gets its own row. Swap the key to find anything repeated: duplicate emails, duplicate order numbers, duplicate enrollments (Project 2 exploits this defensively with a UNIQUE constraint instead).

EXERCISES · CH 9

1. Revenue and order count per order status, biggest revenue first, keeping only statuses with 20+ orders.

▼ SOLUTION

```sql
SELECT status,
       COUNT(*)          AS orders,
       SUM(total_amount) AS revenue
FROM orders
GROUP BY status
HAVING COUNT(*) ≥ 20
ORDER BY revenue DESC;
```

OUTPUT — 6 ROWS — EVERY STATUS CLEARS THE HAVING BAR

status	orders	revenue
delivered	187	2352227.30
confirmed	66	829940.20
shipped	64	804117.55
cancelled	63	789600.35
pending	62	780452.90
processing	58	731104.45

2. Which pincode has the most customers? (Use `address`.)

▼ SOLUTION

```sql
SELECT pincode, COUNT(*) AS customers
FROM address
GROUP BY pincode
ORDER BY customers DESC
LIMIT 1;      -- add ORDER BY pincode as tie-breaker for determinism
```

OUTPUT — 1 ROW

pincode	customers
411001	14

3. Why does
`SELECT gender, AVG(age) FROM maqbool_info WHERE COUNT(*) > 10 GROUP BY gender;` **fail?**

▼ SOLUTION

```
-- WHERE (step 2) runs before grouping; COUNT(*) doesn't exist yet. The
aggregate condition belongs in HAVING.
```

PART III · ANALYZING DATA CHAPTER 10

Subqueries — Queries Inside Queries

Sometimes the filter itself needs a query: "people whose age repeats". Lecture 4 nested a GROUP BY inside a WHERE and it worked like a charm. This chapter formalizes the pattern — subqueries in WHERE, in SELECT, their NULL booby-trap, and when a JOIN is the better tool.

The shape: WHERE col IN (SELECT …)

SQL LECTURE 4

```sql
-- everyone whose age is shared by at least one other person
SELECT * FROM maqbool_info
WHERE age IN (
    SELECT age
    FROM maqbool_info
    GROUP BY age
    HAVING COUNT(age) > 1        -- inner: ages that appear 2+ times
)
ORDER BY age ASC;
```

Read it inside-out: the inner query computes the "wanted list", the outer one filters against it. The inner query runs conceptually first (it has no dependency on outer rows — a **non-correlated** subquery).

MASTERCLASS INSIGHT — DECOMPOSE, THEN COMPOSE

The pro workflow for any nested query: **run the inner query alone**, look at its output (here: a column of repeated ages), *then* wrap it. If the inner result surprises you, the outer query was never your problem. Lecture 4 did exactly this dance — building the GROUP BY first, testing it, and only then embedding it in `WHERE name IN (…)` and its negated twin `WHERE NOT name IN (…)`.

NOT IN — the NULL booby-trap

SQL — THE CLASSIC PRODUCTION BUG

```sql
-- ⚠️ if the inner query can return even ONE NULL,
-- NOT IN returns ZERO rows. Always. Silently.
SELECT * FROM maqbool_info
WHERE age NOT IN (SELECT age FROM maqbool_info GROUP BY age HAVING COUNT(*) >
1);

-- why: age NOT IN (21, NULL, 30) becomes
--      age <> 21 AND age <> NULL AND age <> 30
-- and age <> NULL is UNKNOWN for every row → nothing survives WHERE.

-- the safe forms:
SELECT * FROM maqbool_info
WHERE age NOT IN (SELECT age FROM maqbool_info GROUP BY age HAVING COUNT(*) >
1)
  AND age IS NOT NULL;                        -- guard the outer side

SELECT * FROM maqbool_info m
WHERE NOT EXISTS (SELECT 1 FROM maqbool_info x
                  WHERE x.age = m.age         -- EXISTS ignores NULLs entirely
                    AND x.id <> m.id);        -- (correlated form — see below)
```

Correlated subqueries & scalar subqueries

SQL — PATTERNS

```sql
-- CORRELATED: inner query references the outer row → runs once per outer row.
-- People older than the average age of their own gender:
SELECT name, gender, age
FROM maqbool_info m
WHERE age > (SELECT AVG(age) FROM maqbool_info x
             WHERE x.gender = m.gender);

-- SCALAR: one value, usable anywhere an expression fits:
SELECT name, age,
       (SELECT MAX(age) FROM maqbool_info) AS oldest_in_table
FROM maqbool_info;
```

Subquery or JOIN? A decision table

SITUATION	REACH FOR	WHY
"rows matching a *computed list*"	`IN (SELECT …)`	reads like the question itself
"rows with *no* match" (negation)	`NOT EXISTS` / anti-JOIN (Ch 12)	immune to the NULL trap
"*columns from another table* per row"	JOIN	one pass, index-friendly
"compare each row to an aggregate"	correlated scalar subquery	often clearer than a GROUP BY + JOIN
top-N *per group*	window functions (beyond this guide)	built for exactly this

💡 **PRO TIP**

MySQL 8's optimizer rewrites most `IN (SELECT …)` into semi-joins anyway, so don't fear subqueries for performance — fear ambiguity for correctness. Choose the form a teammate will read correctly at 2 a.m.

EXERCISES · CH 10

1. Customers who have placed at least one order — using a subquery (no JOIN yet).

▼ SOLUTION

```
SELECT id, name FROM maqbool_info
WHERE id IN (SELECT user_id FROM orders);
```

OUTPUT — 5 OF 122 ROWS — EVERYONE EXCEPT THE 5 WHO NEVER ORDERED

```
id    name
____  ________________
1     Arbaz
2     Maqbool
3     Vikram Desai
4     Neha Kulkarni
6     Esha Solanki
… 117 more rows
```

2. The same, but the opposite: customers with no orders — safely.

▼ SOLUTION

```sql
SELECT id, name FROM maqbool_info m
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.user_id = m.id);
-- Project 1 redoes this with LEFT JOIN … IS NULL, the other classic
anti-join.
```

OUTPUT — 5 ROWS — REGISTERED BUT NEVER BOUGHT

id	name
87	Riya Sharma
93	Kabir Menon
100	Isha Verma
111	Dev Raghavan
125	Tara Bose

3. Orders costing more than the store-wide average.

▼ SOLUTION

```sql
SELECT order_number, total_amount
FROM orders
WHERE total_amount > (SELECT AVG(total_amount) FROM orders);
```

OUTPUT — 5 OF 247 ROWS — THE COSTLIER HALF OF THE STORE

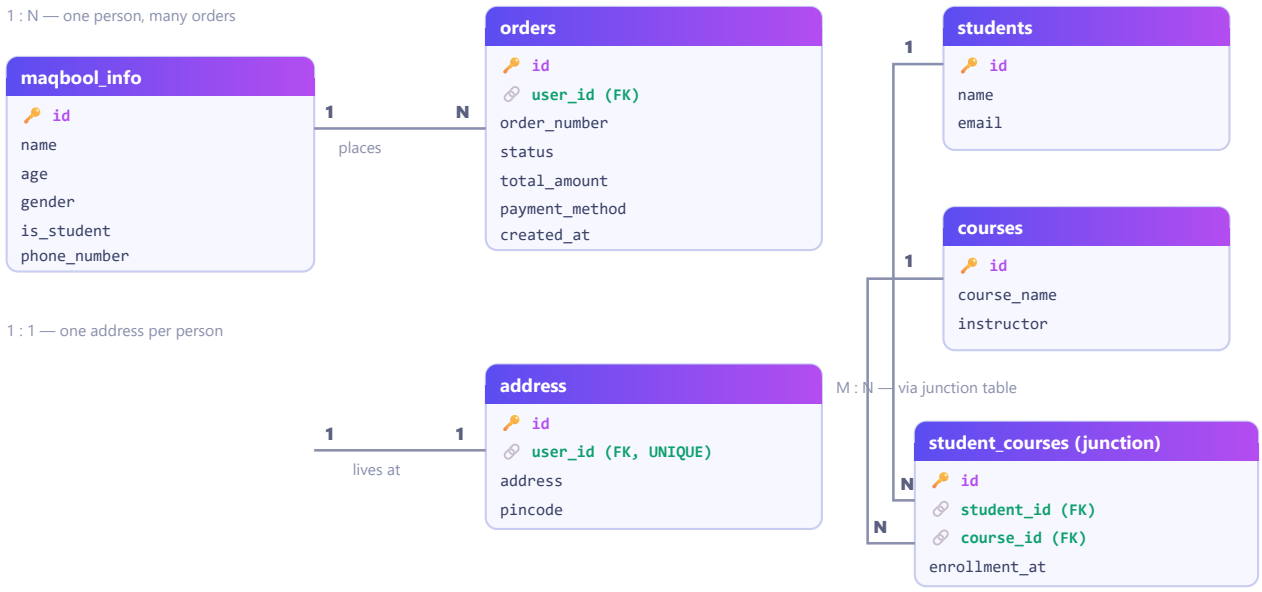
order_number	total_amount
ORD-20260830-0226	24997.60
ORD-20260719-0354	23410.15
ORD-20260901-0007	22988.40
ORD-20260728-0187	17320.50
ORD-20260711-0149	16255.80
… 242 more rows	

PART IV · RELATIONSHIPS CHAPTER 11

Relationships & Foreign Keys

Real data is connected: people place orders, live at addresses, enroll in courses. Lectures 5–7 built all three relationship shapes. This chapter names them, enforces them with foreign keys, and shows the one table trick that makes many-to-many possible.

The three shapes — all present in this dataset



Enforcing it: FOREIGN KEY

Lecture 6 didn't just join tables — it wired them so the *database* refuses orphan rows:

SQL LECTURE 6

```sql
-- the FK columns must exist, be NOT NULL, and match the parent's type:
ALTER TABLE orders  MODIFY user_id INT UNSIGNED NOT NULL;
ALTER TABLE address MODIFY user_id INT NOT NULL;

ALTER TABLE orders
  ADD CONSTRAINT fk_user_orders
  FOREIGN KEY (user_id) REFERENCES maqbool_info(id);

ALTER TABLE address
  ADD CONSTRAINT fk_users_address
  FOREIGN KEY (user_id) REFERENCES maqbool_info(id);

-- prove the wiring — ask the catalog:
SELECT TABLE_NAME, COLUMN_NAME, CONSTRAINT_NAME,
       REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE REFERENCED_TABLE_NAME IS NOT NULL
  AND TABLE_SCHEMA = 'maqbool';
```

MASTERCLASS INSIGHT — REFERENTIAL INTEGRITY

A FK is a live promise: *every child's* `user_id` *points at a real parent*. Insert an order for user 9999 → rejected. Delete user 5 who has orders → rejected (default `RESTRICT`) — protecting you from orphaned data. The trade-off is real though: FK checks cost a little on writes, and legacy/bulk-loading systems sometimes skip FKs and enforce the rule in application code. Know which world you're in.

Many-to-many — the junction table

A student takes many courses; a course holds many students. No column can store that on either side — so Lecture 7 created a third table whose *job* is only to hold the pairs:

SQL LECTURE 7

```sql
CREATE TABLE student_courses (
    id            INT PRIMARY KEY AUTO_INCREMENT,
    student_id    INT NOT NULL,
    course_id     INT NOT NULL,
    enrollment_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES students(id)
                          ON DELETE CASCADE,
    CONSTRAINT fk_course  FOREIGN KEY (course_id)  REFERENCES courses(id)
                          ON DELETE CASCADE,
    UNIQUE (student_id, course_id)     -- one enrollment per pair, ever
);

INSERT INTO student_courses (student_id, course_id) VALUES
(1, 1),  -- Arbaz   → MySQL
(1, 2),  -- Arbaz   → React
(1, 3),  -- Arbaz   → Node.js
(2, 1),  -- Maqbool → MySQL
(2, 3),  -- Maqbool → Node.js
(3, 2);  -- Zara    → React
```

MASTERCLASS INSIGHT — TWO QUIET GENIUSES IN THAT DDL

`ON DELETE CASCADE` : delete a course and its enrollments vanish automatically — no orphans possible. (`RESTRICT` would refuse instead; choose per relationship.)

`UNIQUE(student_id, course_id)` : a composite constraint on the *pair*, not either column alone — Arbaz may take MySQL once, but Arbaz + MySQL twice is now impossible at the engine level. Composite keys are also the answer to "how do I prevent duplicate sign-ups/likes/follows" in any app.

Normalization in 60 seconds

- **1NF** — atomic values: no "phone1, phone2" columns, no comma-separated lists in one cell (why the junction table exists).
- **2NF** — non-key columns depend on the *whole* key (no "instructor name" stored inside `student_courses`).
- **3NF** — non-key columns depend on the key *only* (pincode city is not stored in `address` ; it would drift when pincodes change — compute or join instead).

Your dataset already follows these — which is exactly why queries in Ch 12 need joins to reassemble it.

EXERCISES · CH 11

1. Add a `reviews` table (1 person → many reviews): rating 1–5, FK to `maqbool_info`, cascade on delete.

▼ SOLUTION

```
CREATE TABLE reviews (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    user_id    INT UNSIGNED NOT NULL,
    rating     TINYINT UNSIGNED NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_user_reviews FOREIGN KEY (user_id)
        REFERENCES maqbool_info(id) ON DELETE CASCADE
);
```

OUTPUT — MYSQL SHELL

```
Query OK, 0 rows affected (0.04 sec)
```

2. Try `INSERT INTO orders (user_id, …) VALUES (9999, …)` — read the error. Then check which constraint the catalog query above reports.

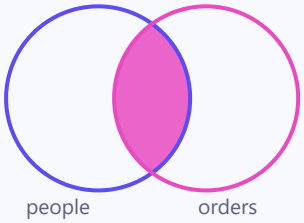
PART IV · RELATIONSHIPS CHAPTER 12

Joins — INNER, LEFT, RIGHT, CROSS

Normalization split the story across tables; joins put it back together. Lectures 5–7 practiced every join type against people, orders and addresses. Learn the two questions each join answers — "who matched?" and "who's missing?" — and 90% of join confusion evaporates.

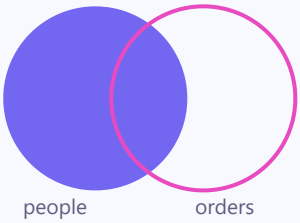
The four join types at a glance

INNER JOIN



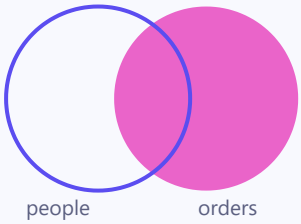
only rows that match in **both** tables

LEFT JOIN



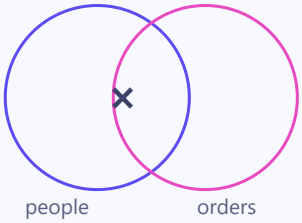
all left rows; NULLs where no match

RIGHT JOIN



all right rows; NULLs where no match

CROSS JOIN



every combination — 127 × 500 rows

INNER JOIN — the workhorse

SQL — LECTURE 5

```sql
SELECT m.id, m.name, m.age,
       o.order_number, o.status, o.total_amount
FROM maqbool_info AS m
INNER JOIN orders AS o ON o.user_id = m.id
ORDER BY m.name ASC
LIMIT 10 OFFSET 0;

-- same query, one filter on the joined table, cleaner with aliases:
SELECT m.name, o.status, o.total_amount
FROM maqbool_info m
INNER JOIN orders o ON o.user_id = m.id
WHERE o.status IN ('delivered', 'confirmed', 'pending')
  AND m.age NOT IN (21, 31, 44)
ORDER BY m.age ASC;
```

MASTERCLASS INSIGHT — WHAT ON REALLY DOES

Conceptually, `FROM a JOIN b ON …` considers row-pairs where the condition holds (the storage engine uses indexes to skip most pairs — a nested-loop over an index, which is why FK columns should be indexed). People with no orders simply produce no qualifying pair, so INNER JOIN **silently drops them**. If "who never ordered?" matters — and it always does — you need LEFT JOIN (next).

LEFT JOIN — keeping the unmatched

SQL — LECTURE 5

```sql
SELECT m.id, m.name, o.order_number, o.status, o.total_amount
FROM maqbool_info m
LEFT JOIN orders o ON o.user_id = m.id;

-- the anti-join: people with NO orders at all
SELECT m.id, m.name
FROM maqbool_info m
LEFT JOIN orders o ON o.user_id = m.id
WHERE o.user_id IS NULL;
```

For a person with no orders, LEFT JOIN still emits their row — with `NULL` in every orders column. That's the mechanism: *left side preserved, missing right side becomes NULL*. Filtering on `o.some_col IS NULL` therefore isolates exactly the no-match rows. This is the **anti-join** — the idiom behind "users with no purchases", "students with no enrollment", "products never ordered".

> ⚠ **WATCH OUT — WHERE CAN SILENTLY UN-LEFT A LEFT JOIN**
>
> `LEFT JOIN orders o … WHERE o.status = 'delivered'` discards the NULL rows — the join might as well have been INNER. To filter the right table *while keeping* unmatched left rows, put the condition in the ON clause: `LEFT JOIN orders o ON o.user_id = m.id AND o.status = 'delivered'`. ON decides *matching*; WHERE decides *surviving*.

RIGHT JOIN, three-table chains, and FULL OUTER

SQL — LECTURES 6 & 7

```sql
-- RIGHT = LEFT with the tables swapped; same engine work:
SELECT m.name, o.order_number, o.status
FROM maqbool_info m
RIGHT JOIN orders o ON m.id = o.user_id;

-- chaining three tables (Lecture 7's flagship query):
SELECT m.name, o.status, o.total_amount, ad.address, ad.pincode
FROM maqbool_info m
JOIN orders  o  ON m.id = o.user_id
JOIN address ad ON m.id = ad.user_id
WHERE o.status = 'processing' AND m.gender = 'male';

-- MySQL has no FULL OUTER JOIN — emulate with UNION (Lecture 5):
SELECT m.id, o.user_id FROM maqbool_info m
LEFT JOIN orders o ON o.user_id = m.id
WHERE m.id IS NOT NULL
UNION
SELECT m.id, o.user_id FROM maqbool_info m
RIGHT JOIN orders o ON m.id = o.user_id;

-- CROSS JOIN: every person × every product-sized table — rare on purpose.
SELECT COUNT(*) FROM maqbool_info CROSS JOIN orders;    -- 127 × 500 = 63,500
```

JOIN	KEEPS	EVERYDAY QUESTION
INNER	matches only	"show orders with their buyer"

`LEFT`	all left + matches (+NULLs)	"all customers, and orders if any" / anti-join
`RIGHT`	all right + matches	rare — usually rewritten as LEFT
`CROSS`	every combination	generating grids, test matrices
`FULL (via UNION)`	everything from both sides	"show unmatched from *both* sides"

💡 **PRO TIP — ALIAS IMMEDIATELY, QUALIFY ALWAYS**

From your first join, write `m`, `o`, `ad` and prefix *every* column. Unqualified `id` in a two-table query is an "ambiguous column" error waiting to happen; qualified names also make a 4-table query readable. Lecture 6's join of `m.id, o.id, o.user_id, a.user_id` was only decipherable because each was aliased (`AS address_id`, `AS address_user_id`).

EXERCISES · CH 12

1. Every student with their course names (triple join through the junction), one row per enrollment.

▼ SOLUTION

```
SELECT s.name AS student, c.course_name, sc.enrollment_at
FROM students s
JOIN student_courses sc ON sc.student_id = s.id
JOIN courses c          ON c.id = sc.course_id
ORDER BY s.name, c.course_name;
```

OUTPUT — 6 ROWS — ONE PER ENROLLMENT

```
student   course_name   enrollment_at
________  ____________  ____________________
Arbaz     MySQL         2026-09-02 10:15:03
Arbaz     Node.js       2026-09-02 10:16:12
Arbaz     React         2026-09-02 10:15:41
Maqbool   MySQL         2026-09-05 16:40:09
Maqbool   Node.js       2026-09-05 16:41:33
Zara      React         2026-09-10 09:22:47
```

2. Courses nobody enrolled in — anti-join.

▼ SOLUTION

```
SELECT c.course_name
FROM courses c
LEFT JOIN student_courses sc ON sc.course_id = c.id
WHERE sc.course_id IS NULL;
```

OUTPUT — EVERY COURSE HAS TAKERS — THE ANTI-JOIN FINDS NOTHING

```
Empty set (0.00 sec)
```

3. People + address + order count per person, including people with zero orders (count must show 0, not disappear).

▼ SOLUTION

```
SELECT m.name, ad.pincode, COUNT(o.id) AS order_count
FROM maqbool_info m
LEFT JOIN address ad ON ad.user_id = m.id
LEFT JOIN orders  o  ON o.user_id  = m.id
GROUP BY m.id, m.name, ad.pincode
ORDER BY order_count DESC;
-- COUNT(o.id) skips NULLs → unmatched people show 0. COUNT(*) would
wrongly show 1.
```

OUTPUT — TOP 5 OF 127 ROWS — AND THE ZEROS AT THE BOTTOM

name	pincode	order_count
Ananya Iyer	411001	16
Rohan Gupta	411014	15
Arbaz	411001	14
Meera Nair	411028	13
Farhan Qureshi	411045	13
… 117 more rows		
Riya Sharma	411032	0
Kabir Menon	411006	0
Isha Verma	411058	0
Dev Raghavan	411020	0
Tara Bose	411014	0

PART V · POWER TOOLS CHAPTER 13

Transactions — COMMIT, ROLLBACK & ACID

Lecture 4 ran an UPDATE, saw the damage, and typed one word to undo it: `ROLLBACK`. That safety net is the transaction — the difference between "I hope my multi-step change works" and "it either all happens or none of it does".

The demo you already ran

SQL — LECTURE 4

```sql
START TRANSACTION;

UPDATE maqbool_info SET age = 23 WHERE name LIKE 'Arbaz%';
SELECT * FROM maqbool_info WHERE name LIKE 'Arbaz%';  -- changed… but only for ME

ROLLBACK;                                             -- ⏪ pretend none of it happened

SELECT * FROM maqbool_info WHERE name LIKE 'Arbaz%';  -- original ages restored

-- and the keep-it version:
START TRANSACTION;
UPDATE maqbool_info SET name = 'Arbaz khan' WHERE name = 'Arbaz';
COMMIT;                                               -- 🔒 now it's permanent
```

1. **START TRANSACTION;** — open a private draft
2. **INSERT / UPDATE / DELETE …** — your session sees changes; others don't — yet

↓ branch ↙

- **3a** **COMMIT;** — ✅ make it permanent & visible to everyone
- **3b** **ROLLBACK;** — ↩️ discard every change since step 1

ACID — the four guarantees

LETTER	GUARANTEE	IN PRACTICE
Atomicity	all or nothing	a transfer = debit + credit; a crash mid-way undoes both
Consistency	constraints hold before & after	a commit can never leave a FK broken or a UNIQUE duplicated
Isolation	concurrent transactions don't trample	your uncommitted draft is invisible to other sessions
Durability	committed = survives power loss	the redo log flushes before COMMIT returns

🧠 **MASTERCLASS INSIGHT — AUTOCOMMIT, THE INVISIBLE DEFAULT**

MySQL commits **every single statement automatically** unless you're inside a transaction. That's why your one-off UPDATEs from earlier chapters were instantly permanent. `START TRANSACTION` suspends autocommit until COMMIT/ROLLBACK. Also crucial for the Ch 3 table: **DDL statements (CREATE/ALTER/DROP/TRUNCATE) cause an implicit commit** — you cannot roll back a `TRUNCATE` even inside a transaction, but you *can* roll back a `DELETE`.

💡 **PRO TIP — WRAP RISKY UPDATE/DELETE IN THE RITUAL**

```
START TRANSACTION;
UPDATE … ;  -- check the affected-rows count matches expectations!
SELECT … ;  -- eyeball the damage before committing
-- happy? COMMIT;  — horrified? ROLLBACK;
```

Long transactions block other writers and bloat undo logs — keep them seconds long, not hours. For checkpoints inside long jobs, use `SAVEPOINT sp1;` / `ROLLBACK TO sp1;`.

EXERCISES · CH 13

1. Simulate a money move: order #5's amount +500, order #9's amount −500 — atomically. Test rollback, then commit for real.

▼ SOLUTION

```
START TRANSACTION;
UPDATE orders SET total_amount = total_amount + 500 WHERE id = 5;
UPDATE orders SET total_amount = total_amount - 500 WHERE id = 9;
SELECT id, total_amount FROM orders WHERE id IN (5, 9);  -- verify
ROLLBACK;   -- rehearsal first!
-- then rerun with COMMIT;
```

OUTPUT — INSIDE THE TRANSACTION … THEN AFTER ROLLBACK

```
id   total_amount        ← inside the transaction (was 4382.75 / 18204.10)
---  ------------
5          4882.75
9         17704.10

Query OK    ← ROLLBACK: both updates discarded

id   total_amount        ← re-check: originals restored
---  ------------
5          4382.75
9         18204.10
```

2. Why did Lecture 4's first `ROLLBACK` after `COMMIT` change nothing?

▼ SOLUTION

```
-- COMMIT made the changes permanent; ROLLBACK only discards work done
since the LAST commit/START TRANSACTION — there was none left to discard.
```

PART V · POWER TOOLS CHAPTER 14

Views — Saved Queries as Virtual Tables

Lecture 7's best join ran over five lines and needed remembering every time. So the lectures saved it: `CREATE VIEW GetALLCustomersData`. A view is a named SELECT the database keeps on file — query it like a table, and the query runs fresh every time.

Create once, query forever

SQL — LECTURE 7

```sql
CREATE VIEW GetALLCustomersData AS
SELECT m.id AS maqbool_id, m.name, m.gender, m.age,
       o.id AS orders_id, o.status, o.item_count, o.total_amount,
       ad.address, ad.pincode
FROM maqbool_info m
JOIN orders  o  ON m.id = o.user_id
JOIN address ad ON m.id = ad.user_id;

-- now the monster query is one line — with WHERE and ORDER BY on top:
SELECT * FROM GetALLCustomersData
WHERE pincode = 411001 AND status = 'confirmed';

SELECT * FROM GetALLCustomersData WHERE gender = 'male';

-- inspect your views:
SHOW FULL TABLES WHERE TABLE_TYPE = 'VIEW';
SHOW CREATE VIEW GetALLCustomersData;
SELECT TABLE_NAME AS view_name, VIEW_DEFINITION
FROM INFORMATION_SCHEMA.VIEWS
WHERE TABLE_SCHEMA = 'maqbool';
```

MASTERCLASS INSIGHT — A VIEW IS A WINDOW, NOT A COPY

No data is stored at CREATE time (it's "virtual"). Every SELECT on the view *re-executes* the underlying query against today's data — insert a new order and the view instantly shows it. Two consequences: views always reflect reality (great), but they are exactly as fast as their underlying query (a slow 5-table join stays slow behind a pretty name — indexes on base tables are what save you, Ch 17). MySQL has no materialized views; for heavy aggregates you'd summarize into a real table on a schedule.

Why views earn their place

- **Simplification** — the triple join, its aliases, its quirks: written once. Consumers just `SELECT`.
- **A stable API** — apps talk to `GetALLCustomersData`; when the schema underneath changes, you repair *one* view instead of hunting every query.
- **Security** — grant access to a view exposing only `name, pincode`, never the base table with phones and ages.
- **Different lenses** — Lecture 7 built `GetCustomerWithOrderAndAddress` with LEFT/RIGHT joins (keeping partially-matched people/addresses) and `GetALLCustomersData` with strict INNER joins (only fully-connected rows). Same data, two deliberate perspectives.

SQL — LECTURE 6 · FIRST VIEW

```sql
CREATE VIEW maqbool_order AS
SELECT m.id, m.age, m.is_student, m.gender, m.name,
       o.status, o.id AS order_id, o.user_id, o.total_amount
FROM maqbool_info m
JOIN orders o ON m.id = o.user_id
WHERE o.status = 'confirmed';

SELECT * FROM maqbool_order LIMIT 10 OFFSET 0;

-- maintain them:
ALTER VIEW maqbool_order AS SELECT … ;    -- replace the definition
DROP VIEW maqbool_order;                  -- remove it (data untouched — there is none)
```

💡 **PRO TIP — SIMPLE VIEWS ARE UPDATABLE**

A view on one table without GROUP BY/DISTINCT can take INSERT/UPDATE/DELETE — the engine routes them to the base table. Handy for exposing a cleaned-up subset to an app while hiding columns. (The join-heavy views above are read-only.)

EXERCISES · CH 14

1. Build `v_enrollments`: student name, course name, instructor, enrollment date — then use it to list Arbaz's schedule in one line.

▼ SOLUTION

```
CREATE VIEW v_enrollments AS
SELECT s.name AS student, c.course_name, c.instructor, sc.enrollment_at
FROM students s
JOIN student_courses sc ON sc.student_id = s.id
JOIN courses c          ON c.id = sc.course_id;

SELECT * FROM v_enrollments WHERE student = 'Arbaz';
```

OUTPUT — CREATE VIEW, THEN THE ONE-LINER

```
Query OK, 0 rows affected (0.01 sec)

student   course_name   instructor   enrollment_at
________  ____________  __________  ____________________

Arbaz     MySQL         Rahul       2026-09-02 10:15:03
Arbaz     Node.js       Priya       2026-09-02 10:16:12
Arbaz     React         Amit        2026-09-02 10:15:41
```

2. Insert a new order, then re-select the view from exercise 1's sibling — confirm the new row appears with zero changes to the view.

PART V · POWER TOOLS CHAPTER 15

CASE & Conditionals — Tiers, Pivots, IFNULL

Raw rows rarely answer business questions — "which customers are Platinum?" does. `CASE` turns conditions into values, and dropped inside an aggregate it becomes a *pivot machine*. Lecture 7's tier labels and status dashboard are the perfect textbook.

CASE — if/else for values

SQL — LECTURE 7 · SLIGHTLY HARDENED

```sql
SELECT m.id AS users_id, o.status, o.total_amount,
    CASE
        WHEN o.total_amount ≤ 5000 THEN 'Silver'
        WHEN o.total_amount ≤ 9000 THEN 'GOLD'
        ELSE 'Platinum'
    END AS customer_tier
FROM orders o
JOIN maqbool_info m ON o.user_id = m.id
ORDER BY total_amount ASC;
```

Cases are checked **top-down, first match wins** — so order your WHENs from specific to general. That's also the bug-fix hiding in this listing: the lecture's original had `WHEN total_amount ≥ 9001 THEN 'Platinum'` with no ELSE, so a 9000.50 order fell through every branch and returned **NULL**. A trailing `ELSE` is cheap insurance — add it unless NULL is truly the answer you want.

CASE inside aggregates — the pivot

SQL — LECTURE 7 · THE STATUS DASHBOARD

```sql
-- four statuses, one row: the classic "wide report"
SELECT
    COUNT(CASE WHEN status = 'confirmed'  THEN 1 END) AS confirmed_count,
    COUNT(CASE WHEN status = 'delivered'  THEN 1 END) AS delivered_count,
    COUNT(CASE WHEN status = 'processing' THEN 1 END) AS processing_count,
    COUNT(CASE WHEN status = 'pending'    THEN 1 END) AS pending_count
FROM orders;

-- revenue split by status, same trick with SUM:
SELECT
    SUM(CASE WHEN status = 'delivered' THEN total_amount ELSE 0 END) AS
delivered_revenue,
    SUM(CASE WHEN status = 'confirmed' THEN total_amount ELSE 0 END) AS
confirmed_revenue
FROM orders;
```

MASTERCLASS INSIGHT — WHY THE NULL-THEN-COUNT DANCE WORKS

Remember Ch 8: `COUNT(expr)` counts rows where expr is **not NULL**. `CASE WHEN status='confirmed' THEN 1 END` yields 1 for confirmed rows and *NULL for everything else* (no ELSE) — so each COUNT sees only its own status. For SUM the roles swap: `ELSE 0` feeds zeros for non-matches. This conditional-aggregation pattern is **the** standard way to build cross-tabs ("rows = product, columns = month, cells = revenue") before you ever meet the `PIVOT`-style fanciness other databases advertise.

NULLFirst aid: IFNULL & friends

SQL — LECTURE 7

```sql
SELECT m.id, m.name, m.gender, m.age,
       IFNULL(m.phone_number, 'no phone') AS phone_number
FROM maqbool_info m;

-- the portable spelling (works in every database):
SELECT m.name, COALESCE(m.phone_number, m.gender, 'unknown') AS fallback_demo
FROM maqbool_info m;      -- returns the FIRST non-NULL among its arguments
```

Use these for *display* and defaults. For *filtering*, keep using `IS NULL` (Ch 6) — and never wrap the indexed column of a WHERE in a function when you can avoid it (Ch 16 explains why).

> ⚠ **WATCH OUT**
>
> `IFNULL(col, 'x')` makes the output a string when col is numeric — fine for reports, wrong for math. And a CASE with mutually exclusive WHENs is a `CHECK`-constraint candidate: if tiers must exist for every amount, consider storing or validating the rule rather than trusting every future query to write its CASE correctly.

EXERCISES · CH 15

1. Age bands in one query: under 20 = 'Gen Z', 20–35 = 'Young adult', over 35 = 'Senior' — with a count per band.

▼ SOLUTION

```
SELECT
    CASE WHEN age < 20 THEN 'Gen Z'
         WHEN age ≤ 35 THEN 'Young adult'
         ELSE 'Senior' END AS age_band,
    COUNT(*) AS people
FROM maqbool_info
GROUP BY age_band      -- MySQL kindly allows the alias here
ORDER BY people DESC;
```

OUTPUT — 3 ROWS

age_band	people
Young adult	87
Senior	22
Gen Z	18

2. Pivot enrollment counts: one row per student, one column per course (MySQL / React / Node.js), 0 where not enrolled.

▼ SOLUTION

```
SELECT s.name,
    SUM(CASE WHEN c.course_name = 'MySQL'   THEN 1 ELSE 0 END) AS
mysql_n,
    SUM(CASE WHEN c.course_name = 'React'   THEN 1 ELSE 0 END) AS
react_n,
    SUM(CASE WHEN c.course_name = 'Node.js' THEN 1 ELSE 0 END) AS node_n
FROM students s
JOIN student_courses sc ON sc.student_id = s.id
JOIN courses c          ON c.id = sc.course_id
GROUP BY s.id, s.name;
```

OUTPUT — 3 ROWS — THE WIDE CROSS-TAB

name	mysql_n	react_n	node_n
Arbaz	1	1	1
Maqbool	1	0	1
Zara	0	1	0

PART V · POWER TOOLS CHAPTER 16

Indexes & EXPLAIN — Making Queries Fast

Everything so far was correctness; this chapter is speed. Lecture 6 built an index, re-ran `EXPLAIN`, and watched the execution plan change shape. Ten minutes of index literacy is worth more than any amount of micro-optimized SQL.

What an index is

Without an index, `WHERE name LIKE 'Arbaz%'` must read **every row** — a "full table scan". An index is a sorted structure (a B-tree) of one column's values with pointers to their rows — the book's back-of-book index. Sorted data means lookups jump straight to the matches instead of reading everything. The bill: indexes consume disk and **slow down writes** slightly (every INSERT/UPDATE must maintain them). Index the columns you *search, join and sort* by — not every column "just in case".

SQL — LECTURE 6

```sql
EXPLAIN SELECT * FROM maqbool_info WHERE name LIKE 'Arbaz%';
-- type: ALL, rows: ~127  → full scan. Fix it:

CREATE INDEX idx_user_name ON maqbool_info(name);
CREATE INDEX idx_user_age  ON maqbool_info(age);

-- composite index — three columns, one structure:
CREATE INDEX multi_idx_user ON maqbool_info(gender, is_student, phone_number);

-- inspect & clean up:
SHOW INDEX FROM maqbool_info;
SELECT TABLE_NAME, INDEX_NAME, NON_UNIQUE, COLUMN_NAME
FROM INFORMATION_SCHEMA.STATISTICS
WHERE TABLE_SCHEMA = 'maqbool' AND INDEX_NAME ≠ 'PRIMARY'
ORDER BY TABLE_NAME;

DROP INDEX idx_user_name ON maqbool_info;   -- (Lecture 6 dropped, recreated — indexes are cheap to rebuild)
```

Reading EXPLAIN without fear

COLUMN	WHAT TO LOOK FOR
`type`	the access path, worst→best: `ALL` (scan everything) → `index` → `range` → `ref` (index lookup) → `eq_ref` / `const` (unique hit). **ALL on a big table = red flag.**
`key`	which index was actually chosen — `NULL` means none
`rows`	estimated rows examined — compare before/after adding an index
`Extra`	`Using index` = answered from the index alone (fast) · `Using filesort` = extra sort pass · `Using where` = filtering after fetch

SQL — LECTURE 6 · BEFORE / AFTER

```sql
EXPLAIN SELECT * FROM maqbool_info WHERE age = 21;
-- before idx_user_age: type=ALL, rows≈127
-- after  idx_user_age: type=ref, key=idx_user_age, rows≈small subset

-- the composite index is LEFTMOST-PREFIX:
EXPLAIN SELECT * FROM maqbool_info
WHERE gender LIKE 'male%' AND age = 21;
-- 'gender' (first column) can seek; 'age' is NOT in this index → still filtered row-by-row.
```

MASTERCLASS INSIGHT — THE LEFTMOST-PREFIX RULE

A composite index is sorted like a phone book sorted by (surname, firstname): useful for surname lookups, surname+firstname lookups — but **useless for firstname alone**. So `(gender, is_student, phone_number)` serves filters on `gender`, `gender+is_student`, or all three — but a query filtering only `phone_number` ignores it entirely. Order composite columns by: equality filters first, range filters last, and put the most selective first. Lecture 6's experiment (`gender LIKE 'male%' AND age = 21` against that index) demonstrated exactly the gap.

⚠ WATCH OUT — HABITS THAT BLIND INDEXES

- Leading-wildcard `LIKE '%arbaz%'` can't seek — `'Arbaz%'` can.
- Wrapping the column in a function: `WHERE YEAR(created_at) = 2026` → prefer a range: `WHERE created_at ≥ '2026-01-01' AND created_at < '2027-01-01'`.
- Implicit type conversion (`WHERE phone_number = 9876543210` against a VARCHAR) casts every row — quote it.
- `UNIQUE` constraints create indexes automatically; the PK is an index too — `user_id` FKs deserve one for join speed.

💡 PRO TIP — MEASURE, CHANGE, RE-MEASURE

The workflow from Lecture 6 scales to production: `EXPLAIN` the slow query → form a hypothesis → create/drop an index → `EXPLAIN` again → keep only if `rows` / `type` improved. On tables with real data you can prefix with `EXPLAIN ANALYZE` to get *actual* timings. Project 3 walks the full loop.

EXERCISES · CH 16

1. Orders are constantly filtered by status. EXPLAIN `WHERE status = 'pending'`, add the index, EXPLAIN again — what changed in `type` and `key`?

▼ SOLUTION

```
EXPLAIN SELECT * FROM orders WHERE status = 'pending';   -- type: ALL
CREATE INDEX idx_orders_status ON orders(status);
EXPLAIN SELECT * FROM orders WHERE status = 'pending';   -- type: ref,
key: idx_orders_status
```

OUTPUT — EXPLAIN · KEY COLUMNS · BEFORE / AFTER

```
id  select_type  table   type  possible_keys      key   rows  Extra
--  ———————————  ——————  ————  ——————————————————  ————  ————  ———————————
1   SIMPLE       orders  ALL   NULL               NULL   500  Using where
← BEFORE

Query OK, 0 rows affected (0.05 sec)   ← CREATE INDEX

id  select_type  table   type  possible_keys      key                 rows
Extra
--  ———————————  ——————  ————  ——————————————————  ——————————————————  ————
———————————
1   SIMPLE       orders  ref   idx_orders_status  idx_orders_status     62
Using index condition   ← AFTER
```

2. Design the ideal composite index for `WHERE user_id = ? AND status = ?` and justify the column order.

▼ SOLUTION

```
CREATE INDEX idx_orders_user_status ON orders(user_id, status);
-- both columns get equality filters; leading with user_id (higher
selectivity: ~500 rows over 127 users)
-- also serves plain "all orders of user X" queries via the prefix rule.
```

OUTPUT EXPLAIN WHERE USER_ID = 42 AND STATUS = 'PENDING' · KEY COLUMNS

```
id  select_type  table   type  possible_keys            key
rows  Extra
--  ___________  ______  ____  _________________________  ___________________
____  ____  ___________

1   SIMPLE       orders  ref   idx_orders_user_status
idx_orders_user_status     1   Using index condition
```

PROJECT 01 E-COMMERCE ANALYTICS

Project 1 — Order Analytics for a Small Store

The brief: you're the first data hire at a shop running on the `maqbool` schema. Founder questions, your queries. Everything from Chapters 5–16 gets used: joins, grouping, CASE, anti-joins, sorting.

Setup — guaranteed data

If you built the notebooks along, `orders` already holds 500 generated rows from Lecture 5. Otherwise, seed a handful of realistic rows so every query below returns something:

SQL SETUP

```sql
USE maqbool;
SELECT COUNT(*) AS orders_ready FROM orders;    -- rows? you're set.
-- (empty? seed a few sample orders:)
INSERT INTO orders (user_id, order_number, status, total_amount, item_count,
payment_method, created_at) VALUES
(1,  'ORD-20260601-0001', 'delivered', 2450.00,  3, 'upi',         NOW() -
INTERVAL 40 DAY),
(2,  'ORD-20260605-0002', 'delivered', 18900.50, 7, 'credit_card', NOW() -
INTERVAL 32 DAY),
(3,  'ORD-20260609-0003', 'shipped',    6200.00,  2, 'cod',         NOW() -
INTERVAL 25 DAY),
(1,  'ORD-20260615-0004', 'pending',    875.25,   1, 'wallet',      NOW() -
INTERVAL 18 DAY),
(4,  'ORD-20260620-0005', 'cancelled', 12300.00,  5, 'debit_card',  NOW() -
INTERVAL 12 DAY),
(2,  'ORD-20260628-0006', 'delivered', 45999.99,  9, 'credit_card', NOW() -
INTERVAL 5  DAY);
```

Sprint 1 — the founder's five questions

Q1 · "HOW MUCH MONEY HAVE WE ACTUALLY MADE ON DELIVERED ORDERS?"

▼ SOLUTION

```
SELECT COUNT(*)      AS delivered_orders,
       SUM(total_amount) AS delivered_revenue,
       AVG(total_amount) AS avg_basket
FROM orders
WHERE status = 'delivered';
```

OUTPUT 1 ROW

delivered_orders	delivered_revenue	avg_basket
187	2352227.30	12579.825188

Q2 · "WHAT DOES OUR ORDER PIPELINE LOOK LIKE — ONE ROW, ALL STATUSES?"

▼ SOLUTION

```
SELECT
    COUNT(*)                                                    AS
total_orders,
    COUNT(CASE WHEN status = 'pending'    THEN 1 END)           AS
pending_n,
    COUNT(CASE WHEN status = 'confirmed'  THEN 1 END)           AS
confirmed_n,
    COUNT(CASE WHEN status = 'processing' THEN 1 END)           AS
processing_n,
    COUNT(CASE WHEN status = 'shipped'    THEN 1 END)           AS
shipped_n,
    COUNT(CASE WHEN status = 'delivered'  THEN 1 END)           AS
delivered_n,
    COUNT(CASE WHEN status = 'cancelled'  THEN 1 END)           AS
cancelled_n,
    ROUND(100 * COUNT(CASE WHEN status = 'delivered' THEN 1 END) /
COUNT(*), 1)
                                                                AS
delivered_pct
FROM orders;
```

OUTPUT THE WHOLE PIPELINE IN ONE ROW

```
total_orders   pending_n   confirmed_n   processing_n   shipped_n
delivered_n   cancelled_n   delivered_pct
____________   _________   ___________   ____________   _________   ________
___   ___________   _______________

500                  62           66            58          64
187           63           37.4
```

Q3 · "WHO ARE OUR TOP 5 CUSTOMERS BY TOTAL SPEND — WITH NAMES?"

▼ SOLUTION

```sql
SELECT m.id, m.name,
       COUNT(o.id)        AS order_count,
       SUM(o.total_amount) AS total_spent
FROM maqbool_info m
JOIN orders o ON o.user_id = m.id
GROUP BY m.id, m.name
ORDER BY total_spent DESC
LIMIT 5;
```

OUTPUT THE LEADERBOARD

```
id    name            order_count   total_spent
____  ______________  ___________   _____________
42    Ananya Iyer              16     201442.25
7     Rohan Gupta              15     193876.40
1     Arbaz                    14     183204.45
118   Meera Nair               13     176915.85
64    Farhan Qureshi           13     171338.90
```

Q4 · "REGISTERED CUSTOMERS WHO NEVER BOUGHT ANYTHING — WHO ARE WE LOSING?" (ANTI-JOIN)

▼ SOLUTION

```sql
SELECT m.id, m.name
FROM maqbool_info m
LEFT JOIN orders o ON o.user_id = m.id
WHERE o.id IS NULL
ORDER BY m.id;
```

OUTPUT 5 ROWS — THE SAME FIVE THE NOT EXISTS FORM FINDS (CH 10)

```
id    name
────  ──────────────
87    Riya Sharma
93    Kabir Menon
100   Isha Verma
111   Dev Raghavan
125   Tara Bose
```

Q5 · "MONTHLY REVENUE TREND FOR THE LAST QUARTER."

▼ SOLUTION

```sql
SELECT DATE_FORMAT(created_at, '%Y-%m') AS month,
       COUNT(*)            AS orders_n,
       SUM(total_amount)   AS revenue
FROM orders
GROUP BY month
ORDER BY month ASC;
```

OUTPUT THE LAST QUARTER

```
month    orders_n   revenue
───────  ─────────  ─────────────
2026-06        166   2088912.30
2026-07        171   2141662.95
2026-08        163   2056867.50
```

Sprint 2 — tiering & the executive one-pager

Q6 · CLASSIFY EVERY CUSTOMER: SILVER UNDER ₹10K, GOLD UNDER ₹30K, PLATINUM ABOVE — AND COUNT EACH TIER.

▼ SOLUTION

```sql
SELECT CASE WHEN spent < 10000  THEN 'SILVER'
            WHEN spent < 30000  THEN 'GOLD'
            ELSE 'PLATINUM' END AS tier,
       COUNT(*) AS customers,
       SUM(spent) AS tier_revenue
FROM (SELECT user_id, SUM(total_amount) AS spent
      FROM orders GROUP BY user_id) AS per_customer   -- derived table (Ch 10)
GROUP BY tier
ORDER BY tier_revenue DESC;
```

OUTPUT — 3 ROWS — WHERE THE MONEY SITS

tier	customers	tier_revenue
PLATINUM	84	5526716.50
GOLD	34	699485.75
SILVER	9	61240.50

Q7 · ONE ROW FOR THE FOUNDER: TOTAL REVENUE, ORDERS, AOV, DISTINCT BUYERS, DELIVERED SHARE, BEST PAYMENT METHOD.

▼ SOLUTION

```sql
SELECT
    SUM(total_amount)                                    AS total_revenue,
    COUNT(*)                                             AS total_orders,
    ROUND(AVG(total_amount), 2)                          AS aov,
    COUNT(DISTINCT user_id)                              AS buyers,
    ROUND(100 * SUM(status = 'delivered') / COUNT(*),1) AS delivered_pct,
-- SUM(boolean) = Ch 8 trick
    (SELECT payment_method FROM orders GROUP BY payment_method
     ORDER BY COUNT(*) DESC LIMIT 1)                     AS top_payment
FROM orders;
```

OUTPUT THE FOUNDER'S ONE LINE

total_revenue	total_orders	aov	buyers	delivered_pct	top_payment
6287442.75	500	12574.89	122	37.4	upi

STRETCH · "OF THE MONEY PEOPLE SPENT, HOW MUCH IS STILL STUCK IN PENDING/PROCESSING?"

▼ SOLUTION

```
SELECT
    SUM(CASE WHEN status IN
('pending','confirmed','processing','shipped')
             THEN total_amount ELSE 0 END) AS money_in_flight,
    SUM(CASE WHEN status = 'cancelled' OR status = 'refunded'
             THEN total_amount ELSE 0 END) AS money_lost
FROM orders;
```

OUTPUT 1 ROW

money_in_flight	money_lost
3145615.10	789600.35

💡 SHIP IT LIKE AN ANALYST

Wrap the sprint-2 answers in `CREATE VIEW exec_summary AS …` and the founder can re-run one line every Monday — Ch 14 in action. Then EXPLAIN your heaviest query (Q5) and decide whether an index on `(status)` or `(created_at)` is the better hire (Ch 16).

PROJECT 02 MANY-TO-MANY SYSTEM

Project 2 — Student Enrollment System

The brief: a coaching institute needs enrollment tracking. You'll design the many-to-many schema properly (Ch 11), load it, report on it (Ch 9, 12, 16), and prove the constraints defend themselves.

Sprint 1 — schema & seed

SQL — SETUP · FROM LECTURE 7

```sql
CREATE DATABASE IF NOT EXISTS institute;
USE institute;

CREATE TABLE students (
    id    INT PRIMARY KEY AUTO_INCREMENT,
    name  VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE
);

CREATE TABLE courses (
    id          INT PRIMARY KEY AUTO_INCREMENT,
    course_name VARCHAR(100) NOT NULL UNIQUE,
    instructor  VARCHAR(100)
);

CREATE TABLE student_courses (
    id            INT PRIMARY KEY AUTO_INCREMENT,
    student_id    INT NOT NULL,
    course_id     INT NOT NULL,
    enrollment_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES students(id) ON
DELETE CASCADE,
    CONSTRAINT fk_course  FOREIGN KEY (course_id)  REFERENCES courses(id)  ON
DELETE CASCADE,
    UNIQUE (student_id, course_id)
);

INSERT INTO students (name, email) VALUES
('Arbaz',   'arbaz@test.com'),
('Maqbool', 'maqbool@test.com'),
('Zara',    'zara@test.com');

INSERT INTO courses (course_name, instructor) VALUES
('MySQL',   'Rahul'),
('React',   'Amit'),
('Node.js', 'Priya');

INSERT INTO student_courses (student_id, course_id) VALUES
(1,1),(1,2),(1,3),    -- Arbaz: all three
(2,1),(2,3),          -- Maqbool: MySQL, Node.js
(3,2);                -- Zara: React
```

Sprint 2 — the reports

R1 · FULL ROSTER: WHO STUDIES WHAT, WITH INSTRUCTOR — NEWEST ENROLLMENTS FIRST.

▼ SOLUTION

```
SELECT s.name AS student, c.course_name, c.instructor, sc.enrollment_at
FROM students s
JOIN student_courses sc ON sc.student_id = s.id
JOIN courses c          ON c.id = sc.course_id
ORDER BY sc.enrollment_at DESC, s.name;
```

OUTPUT — 6 ROWS

student	course_name	instructor	enrollment_at
Zara	React	Priya	2026-09-10 09:22:47
Maqbool	Node.js	Priya	2026-09-05 16:41:33
Maqbool	MySQL	Rahul	2026-09-05 16:40:09
Arbaz	Node.js	Priya	2026-09-02 10:16:12
Arbaz	React	Amit	2026-09-02 10:15:41
Arbaz	MySQL	Rahul	2026-09-02 10:15:03

R2 · COURSE POPULARITY — INCLUDING COURSES WITH ZERO STUDENTS (LEFT JOIN FROM COURSES).

▼ SOLUTION

```
SELECT c.course_name, c.instructor,
       COUNT(sc.id) AS enrolled
FROM courses c
LEFT JOIN student_courses sc ON sc.course_id = c.id
GROUP BY c.id, c.course_name, c.instructor
ORDER BY enrolled DESC;
```

OUTPUT — 3 ROWS — A DEAD HEAT, AND NO ZERO ROW THIS TIME

course_name	instructor	enrolled
MySQL	Rahul	2
Node.js	Priya	2
React	Amit	2

R3 · EVERYONE NOT IN THE MYSQL COURSE — TWO WAYS: NOT EXISTS AND LEFT JOIN. PROVE BOTH AGREE.

▼ SOLUTION

```sql
SELECT s.name FROM students s
WHERE NOT EXISTS (
    SELECT 1 FROM student_courses sc
    JOIN courses c ON c.id = sc.course_id
    WHERE sc.student_id = s.id AND c.course_name = 'MySQL'
);

SELECT s.name FROM students s
JOIN student_courses sc ON sc.student_id = s.id
JOIN courses c ON c.id = sc.course_id
WHERE c.course_name = 'MySQL'
-- students IN that inner list, negated:
RIGHT JOIN students s2 ON s2.id = s.id  -- (or simpler: WHERE s.id NOT IN (…))
WHERE s.id IS NULL;
-- clearest third option:
SELECT s.name FROM students s
WHERE s.id NOT IN (SELECT sc.student_id FROM student_courses sc
                   JOIN courses c ON c.id = sc.course_id
                   WHERE c.course_name = 'MySQL');
```

OUTPUT — ALL THREE FORMS RETURN THE SAME SINGLE ROW

```
name
_______
Zara
```

R4 · ENROLLMENT MATRIX — STUDENTS AS ROWS, COURSES AS COLUMNS (CH 15 PIVOT).

▼ SOLUTION

```sql
SELECT s.name,
       SUM(c.course_name = 'MySQL')   AS mysql_n,
       SUM(c.course_name = 'React')   AS react_n,
       SUM(c.course_name = 'Node.js') AS node_n,
       COUNT(*)                       AS total
FROM students s
JOIN student_courses sc ON sc.student_id = s.id
JOIN courses c          ON c.id = sc.course_id
GROUP BY s.id, s.name;
```

OUTPUT — 3 ROWS — SAME NUMBERS AS THE CASE VERSION, SHORTER SQL

name	mysql_n	react_n	node_n	total
Arbaz	1	1	1	3
Maqbool	1	0	1	2
Zara	0	1	0	1

Sprint 3 — watch the constraints work

SQL — ATTACK THE SCHEMA · READ THE ERRORS

```sql
-- ① duplicate enrollment → Error 1062 (composite UNIQUE)
INSERT INTO student_courses (student_id, course_id) VALUES (1, 1);

-- ② enrollment for a ghost student → Error 1452 (FK)
INSERT INTO student_courses (student_id, course_id) VALUES (999, 1);

-- ③ two Zara emails → Error 1062 (UNIQUE on students.email)

-- ④ the CASCADE promise: remove a course…
DELETE FROM courses WHERE course_name = 'React';
SELECT * FROM student_courses;              -- …its enrollments are gone too

-- ⑤ restore for re-runs:
INSERT INTO courses (id, course_name, instructor) VALUES (2, 'React', 'Amit');
INSERT INTO student_courses (student_id, course_id) VALUES (1,2),(3,2);
```

MASTERCLASS INSIGHT — WHAT YOU JUST PROVED

You enforced four business rules with **zero application code**: “one enrollment per student-course”, “no ghost students”, “unique emails”, “no orphan enrollments”. Every future frontend bug, script typo or manual fix hits these walls instead of your data. This is what “the database defends itself” means — and it's why interviewers love asking you to design a junction table.

PROJECT 03 PERFORMANCE TUNE-UP

Project 3 — Query Performance Tune-up

The brief: the app feels slow "sometimes". Your job: measure, index, re-measure, and write the report a senior would sign off. You'll exercise every EXPLAIN skill from Chapter 16 — with a bigger dataset so differences are visible.

Step 1 — scale up the lab

SQL — SETUP · 5,000-ROW COPY OF MAQBOOL_INFO

```sql
USE maqbool;
CREATE TABLE big_users AS
SELECT m.* FROM maqbool_info m
CROSS JOIN (SELECT 1 UNION SELECT 2 UNION SELECT 3 UNION SELECT 4
            UNION SELECT 5 UNION SELECT 6 UNION SELECT 7 UNION SELECT 8
            UNION SELECT 9 UNION SELECT 10 UNION SELECT 11 UNION SELECT 12
            UNION SELECT 13 UNION SELECT 14 UNION SELECT 15 UNION SELECT 16
            UNION SELECT 17 UNION SELECT 18 UNION SELECT 19 UNION SELECT 20
            UNION SELECT 21 UNION SELECT 22 UNION SELECT 23 UNION SELECT 24
            UNION SELECT 25 UNION SELECT 26 UNION SELECT 27 UNION SELECT 28
            UNION SELECT 29 UNION SELECT 30 UNION SELECT 31 UNION SELECT 32
            UNION SELECT 33 UNION SELECT 34 UNION SELECT 35 UNION SELECT 36
            UNION SELECT 37 UNION SELECT 38 UNION SELECT 39 UNION SELECT 40) f;
-- note: CREATE TABLE … AS SELECT copies data but NOT indexes/constraints.

SELECT COUNT(*) FROM big_users;    -- ≈ 5,080 rows
```

Step 2 — the before/after loop

T1 · BASELINE: FIND ONE PERSON BY NAME. RECORD `TYPE` AND `ROWS`.

▼ SOLUTION

```sql
EXPLAIN SELECT * FROM big_users WHERE name = 'Arbaz Khan';
-- BEFORE: type = ALL (full scan of ~5,080 rows), key = NULL
```

OUTPUT EXPLAIN · KEY COLUMNS

```
id  select_type  table      type  possible_keys  key   rows  Extra
--  ___________  _________  ____  _____________  ____  ____  __________
1   SIMPLE       big_users  ALL   NULL           NULL  5080  Using where
```

T2 · ADD THE INDEX, RE-RUN, RECORD THE NEW PLAN.

▼ SOLUTION

```
CREATE INDEX idx_big_name ON big_users(name);
EXPLAIN SELECT * FROM big_users WHERE name = 'Arbaz Khan';
-- AFTER: type = ref, key = idx_big_name, rows = the actual match count
(~160)
```

OUTPUT EXPLAIN · KEY COLUMNS · AFTER THE INDEX

```
id  select_type  table      type  possible_keys  key           rows  Extra
--  ___________  _________  ____  _____________  ____________  ____  _______
_____________

1   SIMPLE       big_users  ref   idx_big_name   idx_big_name   160  Using
index condition
```

T3 · RANGE CHECK: AGES 30–35 — THEN MAKE IT USE THE LEFTMOST PREFIX OF A COMPOSITE.

▼ SOLUTION

```
EXPLAIN SELECT * FROM big_users WHERE age BETWEEN 30 AND 35;
CREATE INDEX idx_big_gender_age ON big_users(gender, age);
EXPLAIN SELECT * FROM big_users WHERE gender = 'female' AND age BETWEEN
30 AND 35;
-- gender = equality (seeks), age = range (scans within) → textbook
composite order.
-- Now the trap:
EXPLAIN SELECT * FROM big_users WHERE age BETWEEN 30 AND 35;
-- age alone → index ignored (leftmost 'gender' missing) → type = ALL
again.
```

OUTPUT EXPLAIN · KEY COLUMNS · THE COMPOSITE AT WORK, THEN IGNORED

```
id  select_type  table      type   possible_keys        key
rows  Extra
--  ___________  _________  _____  ___________________  ___________________
____  ________________________

1   SIMPLE       big_users  range  idx_big_gender_age   idx_big_gender_age
96  Using index condition   ← gender = + age range

id  select_type  table      type   possible_keys        key
rows  Extra
--  ___________  _________  _____  ___________________  ___________________
____  ____________

1   SIMPLE       big_users  ALL    idx_big_gender_age   NULL
5080  Using where             ← age alone: ignored
```

T4 · THE WILDCARD TRAP FROM CH 16 — DEMONSTRATE IT.

▼ SOLUTION

```sql
EXPLAIN SELECT * FROM big_users WHERE name LIKE 'Arbaz%';   -- uses
idx_big_name (prefix seek)
EXPLAIN SELECT * FROM big_users WHERE name LIKE '%Arbaz%';  -- type = ALL
— leading % blinds the index
```

OUTPUT EXPLAIN · KEY COLUMNS · PREFIX SEEK VS BLINDED SCAN

```
id  select_type  table      type  possible_keys  key           rows  Extra
--  ___________  _________  ____  _____________  ____________  ____  _______
_________________

1   SIMPLE       big_users  ref   idx_big_name   idx_big_name   880  Using
index condition   ← 'Arbaz%'

id  select_type  table      type  possible_keys  key           rows  Extra
--  ___________  _________  ____  _____________  ____________  ____  _______
____

1   SIMPLE       big_users  ALL   NULL           NULL          5080  Using
where             ← '%Arbaz%'
```

T5 · CLEANUP — AND A CONFESSION ABOUT WRITE COST.

▼ SOLUTION

```
SHOW INDEX FROM big_users;
DROP INDEX idx_big_name ON big_users;
DROP INDEX idx_big_gender_age ON big_users;
-- time an INSERT loop before/after indexes to feel the write-tax
yourself:
-- INSERT INTO big_users (name, age, gender) SELECT name, age, gender
FROM maqbool_info;  (×N)
```

OUTPUT — MYSQL SHELL · TWO INDEXES DROPPED

```
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Step 3 — write the tuning report

🧠 THE FOUR-LINE FORMAT SENIORS EXPECT

Query: the SQL. **Before:** type/rows/key from EXPLAIN. **Change:** the index DDL (or code fix). **After:** new type/rows/key + why the optimizer picked it. Copy that block into your repo next to the query — future-you inherits the reasoning, not just the index.

💡 WHERE TO GO FROM HERE

Re-run the loop on `orders` with `EXPLAIN ANALYZE` (real timings), try a covering index (`Extra: Using index`), and time everything with `SET profiling = 1;` or the `sys` schema views you met on page 4.

REFERENCE ONE PAGE TO RULE THEM ALL

Cheatsheet & What's Next

Every statement from the course, stripped to the bone. Screenshot this page.

SCHEMA — DDL (CH 2–3)

```
CREATE DATABASE db;            USE db;
CREATE TABLE t (id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50) NOT NULL, price DECIMAL(10,2),
  status ENUM('a','b') DEFAULT 'a',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);
DESC t;                        SHOW TABLES;
ALTER TABLE t ADD col TYPE;
ALTER TABLE t MODIFY col TYPE;
ALTER TABLE t DROP COLUMN col;
TRUNCATE TABLE t;              DROP TABLE t;
```

DATA — DML (CH 4, 13)

```
INSERT INTO t (a,b) VALUES (1,'x'), (2,'y');
UPDATE t SET b = 'z' WHERE id = 5;      -- SELECT first!
DELETE FROM t WHERE id = 5;
START TRANSACTION;  -- …statements…  COMMIT; / ROLLBACK;
```

THE UNIVERSAL QUERY (CH 5–9)

```
SELECT DISTINCT cols / aggregates
FROM t
JOIN u ON u.fk = t.pk          -- 1: gather
WHERE row_condition            -- 2: filter rows
GROUP BY cols                  -- 3: bucket
HAVING group_condition         -- 4: filter buckets
ORDER BY col [ASC|DESC]        -- 6: sort
LIMIT n OFFSET m;              -- 7: window
```

FILTER TOOLBOX (CH 6)

```
WHERE a = 1 AND (b = 2 OR c = 3)
WHERE col IN (1,2,3)   / NOT IN (…)
WHERE col BETWEEN 10 AND 20
WHERE col LIKE 'ab%'  '%ab'  '%ab%'  '_b%'
WHERE col IS NULL / IS NOT NULL    -- never = NULL
```

AGGREGATES (CH 8–9, 15)

```
COUNT(*) · COUNT(col) · COUNT(DISTINCT col)
SUM · MIN · MAX · AVG              -- skip NULLs
CASE WHEN cond THEN x [WHEN…] [ELSE y] END
COUNT(CASE WHEN s='ok' THEN 1 END)   -- pivot count
SUM(CASE WHEN s='ok' THEN amt ELSE 0 END)
IFNULL(col, 'fallback')  /  COALESCE(a,b,c)
```

JOINS (CH 11–12)

```
INNER JOIN u ON u.fk = t.pk      -- matches only
LEFT  JOIN u ON …                -- keep all left
LEFT JOIN u ON … WHERE u.id IS NULL   -- anti-join
CROSS JOIN u                     -- every pair
FULL OUTER ≈ LEFT … UNION RIGHT …
3+ tables: JOIN a→b→c, alias every table.
```

VIEWS & KEYS (CH 11, 14)

```
CREATE VIEW v AS SELECT …;    DROP VIEW v;
FOREIGN KEY (fk) REFERENCES p(id) ON DELETE CASCADE
UNIQUE (a, b)   -- composite guard
```

PERFORMANCE (CH 16)

```
EXPLAIN SELECT …;            -- read type/key/rows
CREATE INDEX ix ON t(col);
CREATE INDEX ix ON t(a, b);  -- leftmost prefix!
SHOW INDEX FROM t;           DROP INDEX ix ON t;
-- blinders: '%x%', functions on columns, type mismatch
```

Graduation map — what to learn next

✓ **Window functions** — `ROW_NUMBER() OVER (PARTITION BY …)`: top-N per group, running totals.

- ✓ **Stored procedures & functions** — reusable server-side logic.
- ✓ **Triggers & events** — automatic reactions to changes.
- ✓ **Users & permissions** — `GRANT` / `REVOKE`, least privilege.

- ✓ **Backup & recovery** — `mysqldump`, binlogs. Practice restoring once.
- ✓ **Transactions in depth** — isolation levels, locking, deadlocks.
- ✓ **Query optimization** — `EXPLAIN ANALYZE`, the `sys` schema.
- ✓ **Beyond MySQL** — PostgreSQL, and how ORMs translate your SQL.

🎓 **YOU MADE IT**

Sixteen chapters ago, `SHOW DATABASES;` was the whole world. You can now design schemas with real constraints, reshape data with joins and subqueries, guard it with transactions, present it with views and CASE-driven reports, and make it fast with indexes — then prove it all in three projects. Keep the notebooks; rerun a chapter a week. **The database defends itself — and now, so do you.**